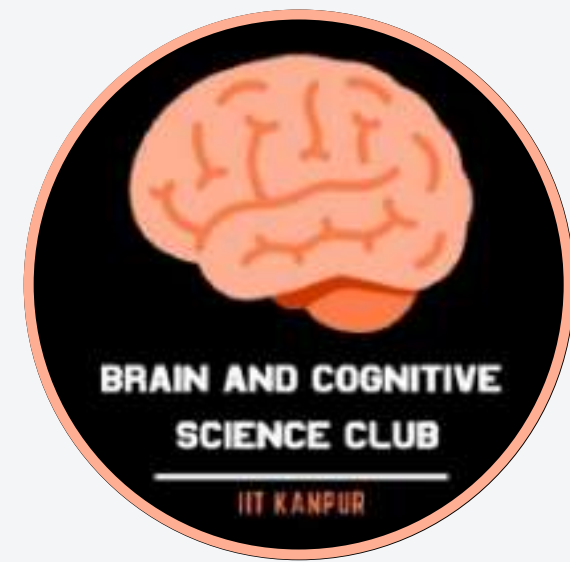
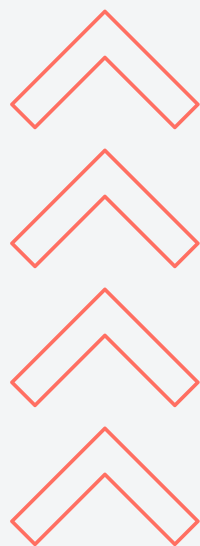




DREAM DIFFUSION

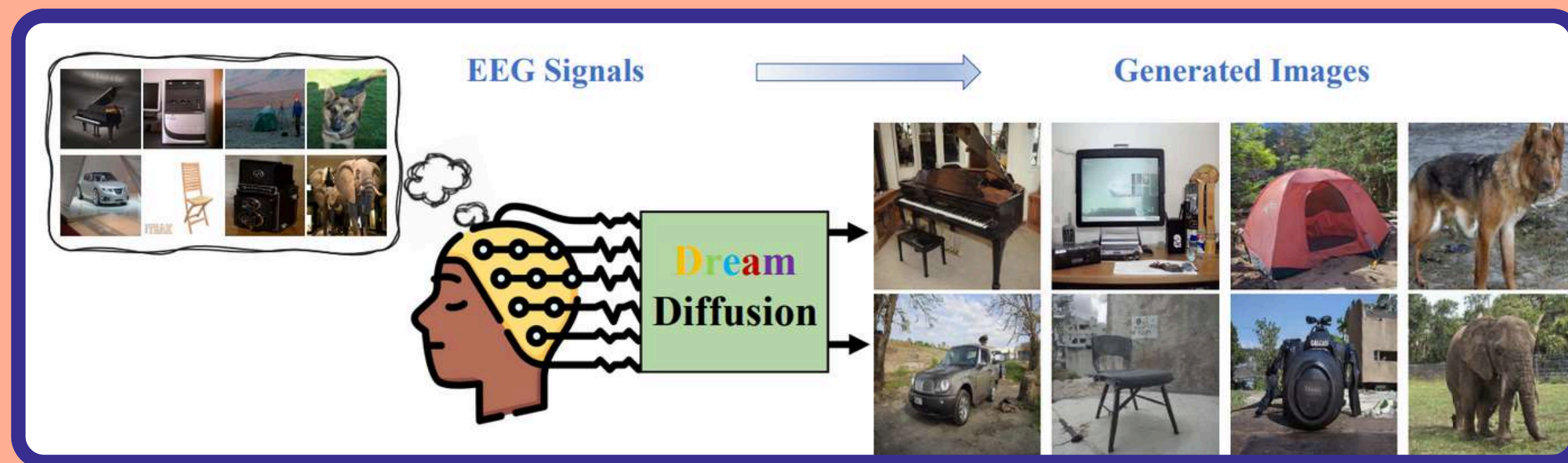
EndTerm Presentation



PROJECT OVERVIEW

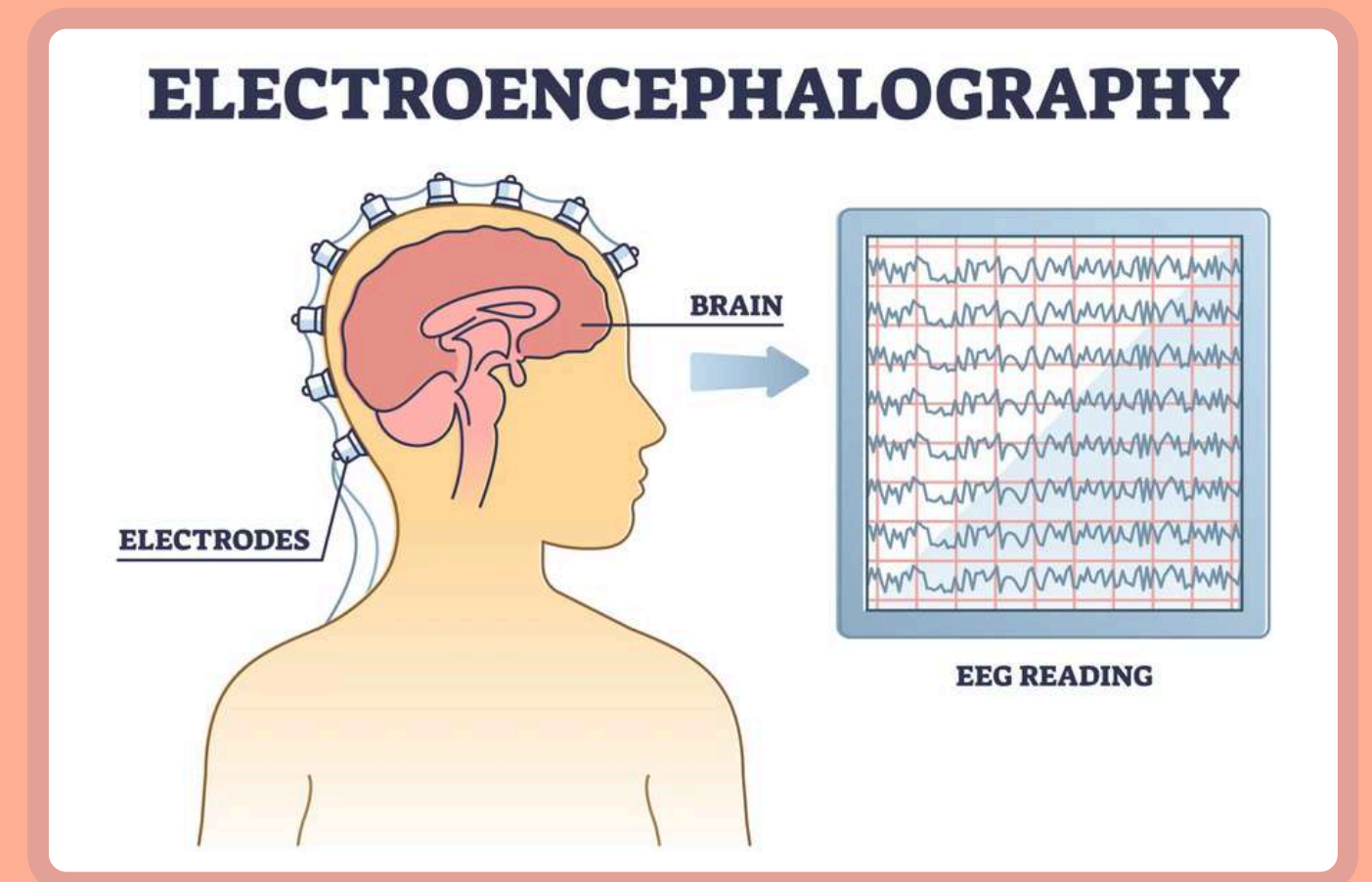
The project aims to translate brain activity (**EEG** signals) into realistic images using state-of-the-art deep learning and generative models.

We are focused in building an end-to-end pipeline that enables the reconstruction of visual stimuli from EEG signals by integrating neuroscience with modern AI tools like **CLIP** and **Stable Diffusion**.



EEG

- The EEG is an electrophysiological technique for the recording of electrical activity arising from the human brain.
- Small flat metal discs called electrodes are attached to your scalp with wires. The electrodes analyse the electrical impulses in your brain and send signals to a computer that records the results.



PYSCHOPY

- We are using PsychoPy to present a set of images along with an attentional check mechanism. The experiment includes 10 image classes, each containing 5 images, displayed one at a time.
- After each image, a cross ('+') appears at a random screen location as an attentional check. The participant must click at the centre of the cross to proceed to the next image.
- To ensure precise data collection, we use Lab Streaming Layer (LSL) to send event markers at the start and end of each image presentation. This allows us to capture data only during the actual image display period. A dedicated marker stream is created using PyLSL for this purpose.



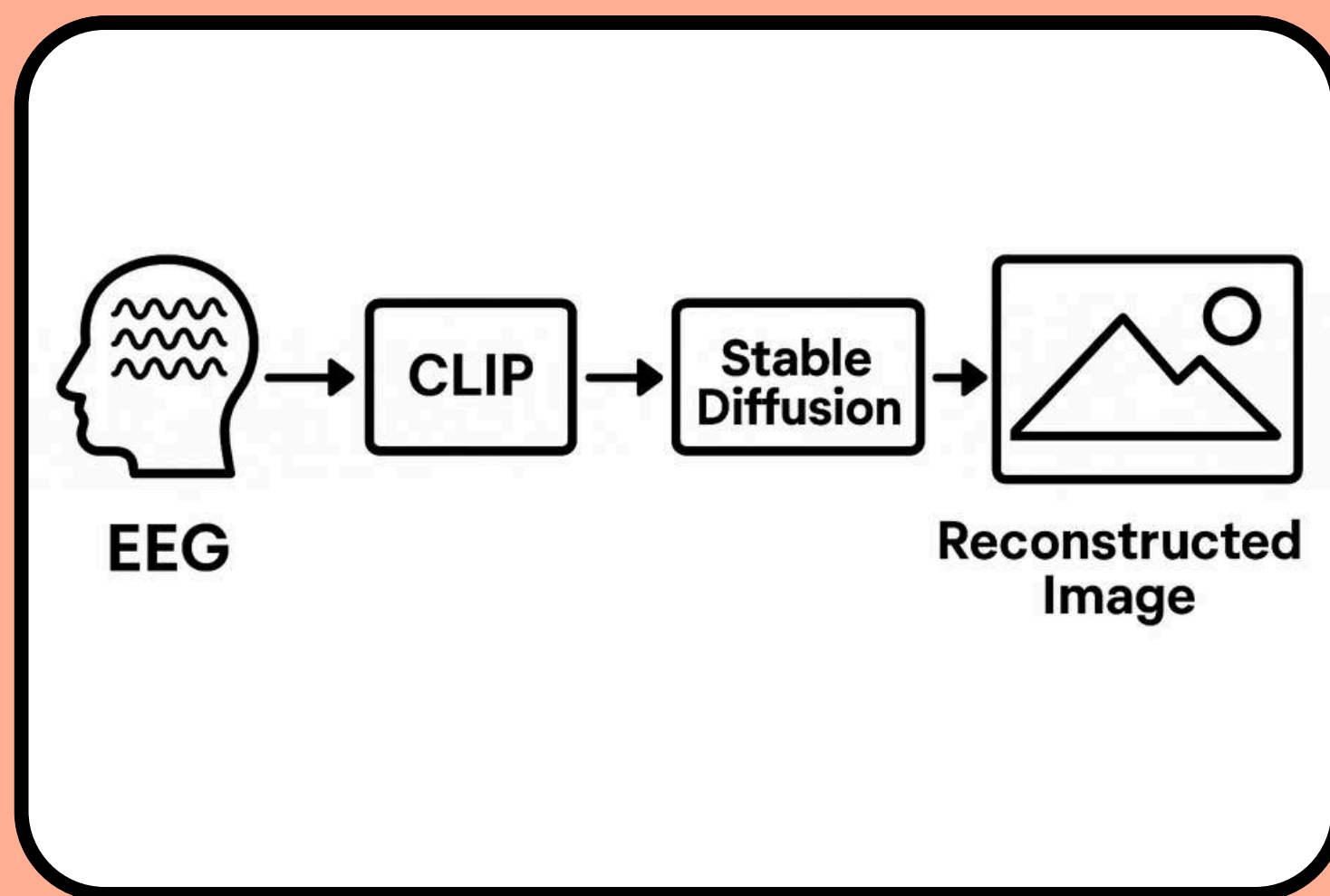
PYGAME

- We implement our image-presentation paradigm in Pygame, where each of our 10 stimulus classes (5 images per class) is loaded as a Surface and “blitted” to the screen. Immediately after each stimulus, we draw a small cross randomly and wait for a `MOUSEBUTTONDOWN` event within the cross’s bounding box before proceeding.
- To guarantee precise timing, we call `pygame.display.flip()` right after each blit to push the image (and later the cross) to the display buffer, and we timestamp the exact stimulus frames.
- Meanwhile, Lab Streaming Layer (LSL) markers—sent via a dedicated PyLSL StreamOutlet—mark the onset and offset of each image presentation.
- Pygame’s tight game-loop architecture and low-level control over the display buffer make it a lightweight yet powerful alternative to PsychoPy for custom-built, Pythonic experiments.

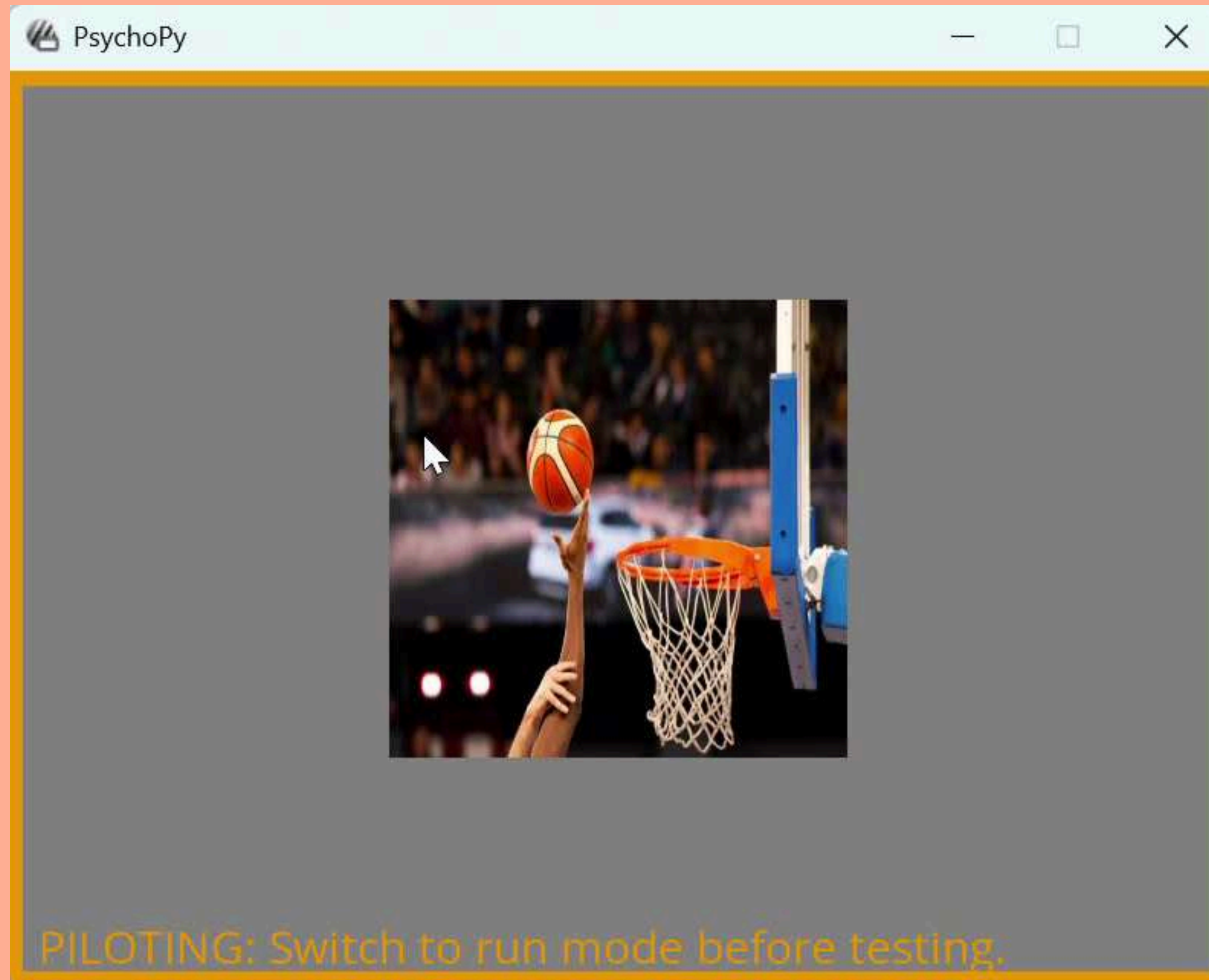


PARADIGM

In our experiment, each participant saw 50 images, drawn from 10 different categories, with five photos per category. Every trial follows the same timeline: the image on screen for 5s, an attentional check in the form of a randomly placed '+' that the participant must click before the subsequent trial begins. We record EEG only during the 5s image presentation.



PSYCHOPY EXPERIMENT



DATA COLLECTION



DATA COLLECTION



Why Attentional Checks?

We include attentional checks to ensure the participant focuses on each image. The EEG signals become **noisy and meaningless** if someone looks away or gets distracted. We confirm that every trial reflects genuine visual attention by requiring a click on a randomly positioned cross after each image.



Sending Event Markers (LSL)

In PsychoPy, we use pylsl to create a marker stream named 'MarkerStream'. When an image appears, we push a sample containing the image's class ID. We push another sample containing class ID + 10 just before the attentional check begins. These markers let us align EEG data with stimulus onset and offset down to the millisecond.

```
Image animal.jpg started at 28.896 seconds  
Image animal.jpg ended at 34.373 seconds  
Duration shown: 5.476 seconds  
Image sport.jpg started at 35.704 seconds  
Image sport.jpg ended at 41.191 seconds  
Duration shown: 5.487 seconds  
Image person.jpg started at 42.837 seconds  
Image person.jpg ended at 48.292 seconds  
Duration shown: 5.455 seconds
```


EEG Recording Setup

For EEG data collection, we used a headset with 4 electrodes. The device supports LSL streaming and sends 8-bit float data over WebSocket.

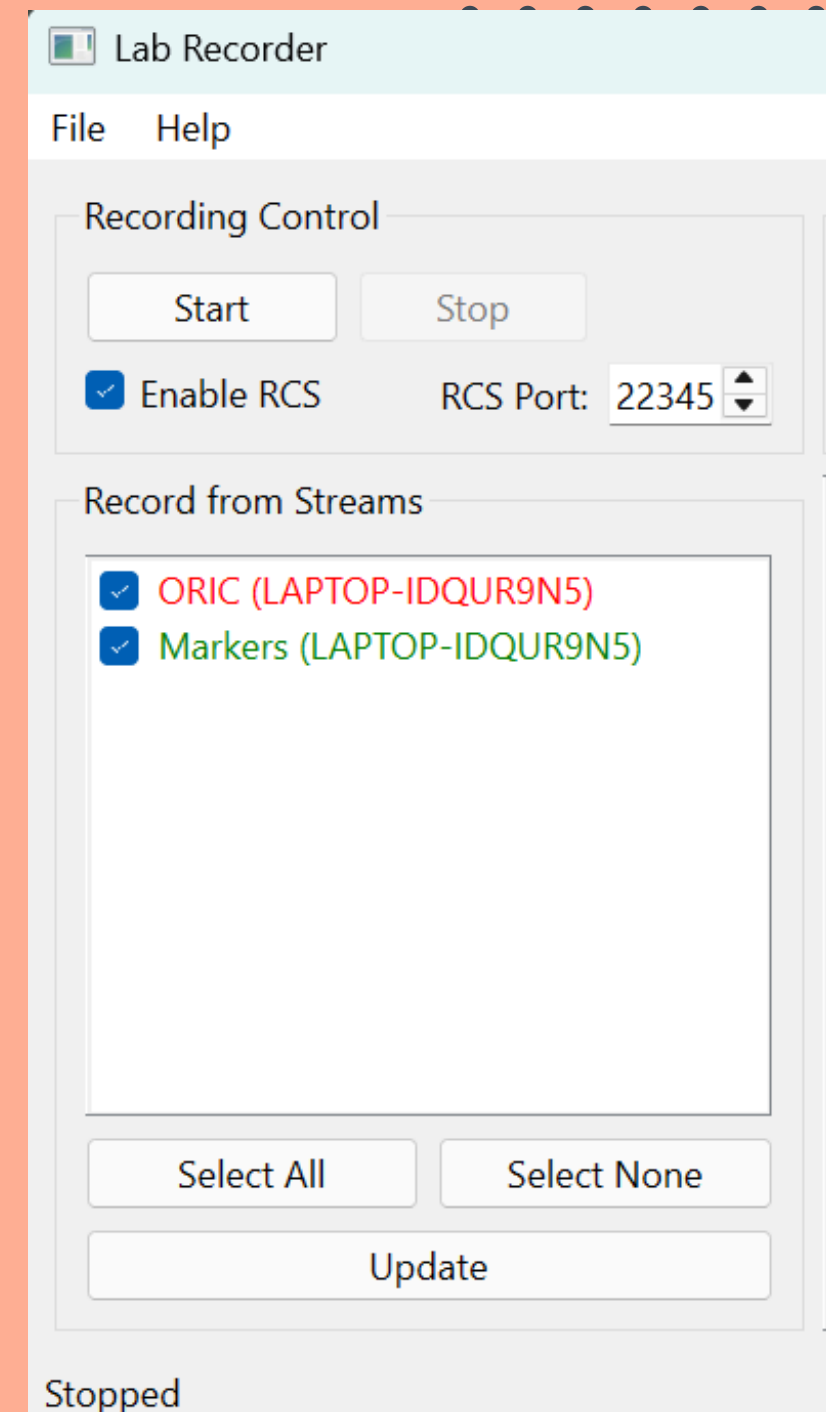
We used LabRecorder to capture the EEG signal in real-time. It records two streams:

- The EEG data stream named ORIC
- The Marker stream from PsychoPy

Both are recorded together into a single .xdf file for precise time synchronization between stimulus and brain response

EEG Streaming with pc_mark6.py

Our pc_mark6.py script connects to the EEG amplifier over WebSocket, reads eight channels of data at 250 Hz, and pushes each sample into an LSL stream called 'ORIC'. The script continuously logs frames-per-second, sample loss, and duplicates to ensure data integrity in real time.



EEG Signal Visualization & Filtering

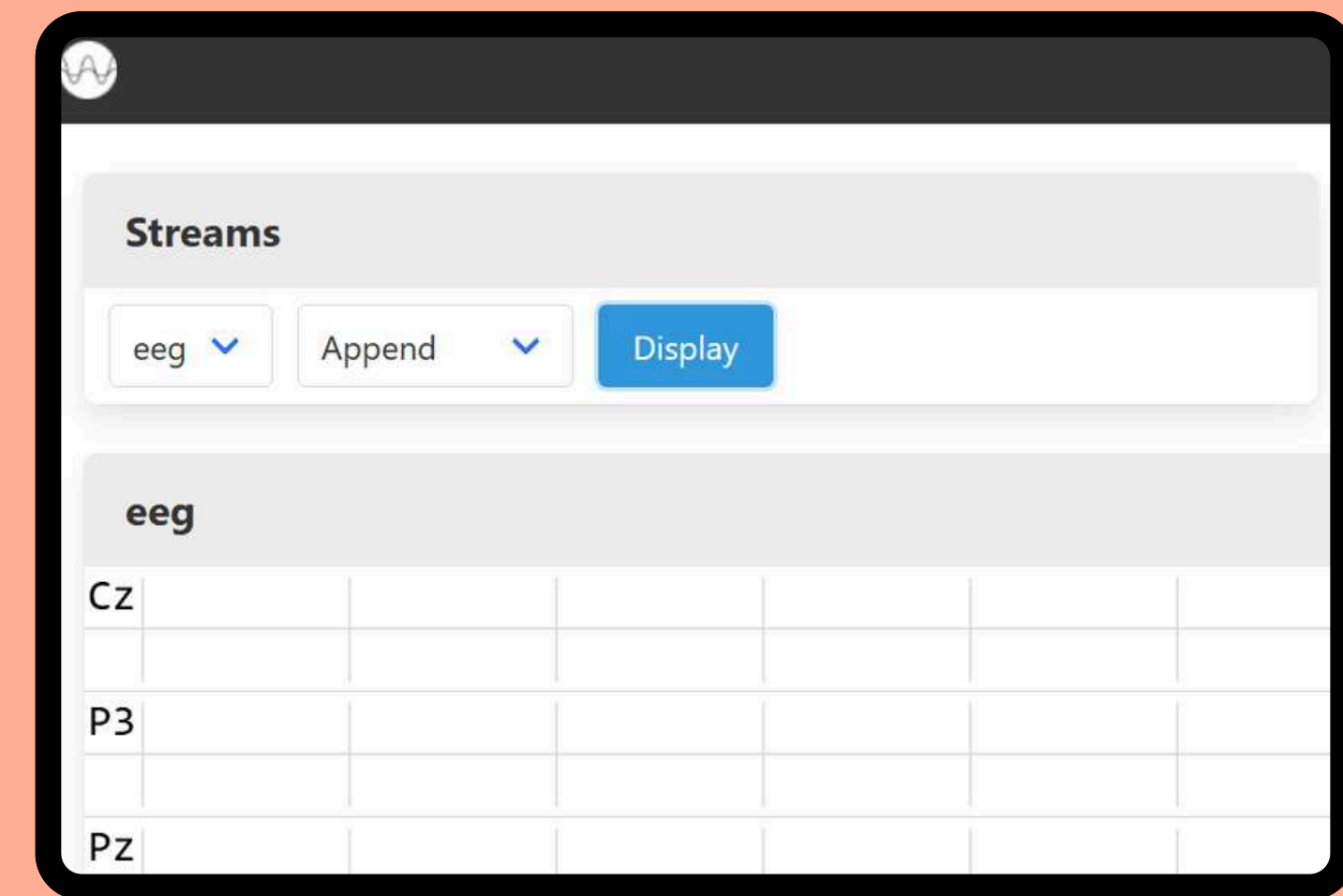
We used Timeflux to visualise EEG signals in real time. The data was streamed via LSL from a source named 'ORIC', which provided signals from 8 channels.

To ensure signal quality, we applied a notch filter to eliminate powerline noise and a bandpass filter (5-95 Hz) to retain relevant brain activity frequencies. The filtered EEG signals were displayed live using the Timeflux UI, enabling real-time monitoring and quality checks during the experiment.

After the experiment, all EEG and marker data were saved in a single .xdf file using LabRecorder.

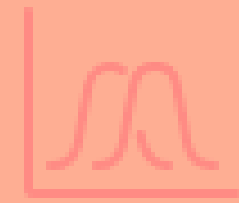
To access and analyse this data, we used the pyxdf Python library. It allowed us to load both the EEG signal and the event markers in a structured format

TIMEFLUX UI

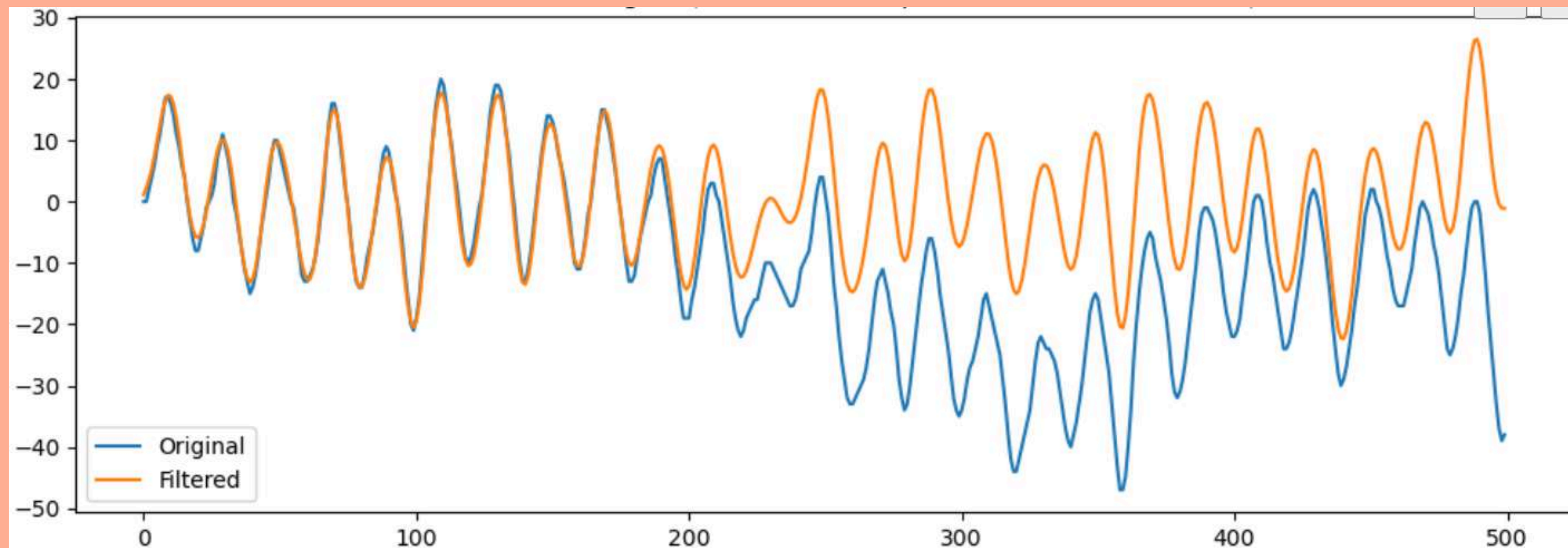


DATA PREPROCESSING

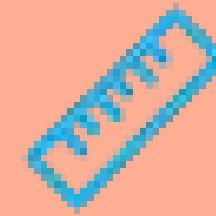
1. BANDPASS FILTERING



Isolates a specific frequency range (5-95 Hz), effectively eliminating low-frequency artefacts (e.g., DC offset from electrode-skin interface) and high-frequency noise (e.g., electromagnetic interference)



2. RESAMPLING TO MEDIAN LENGTH



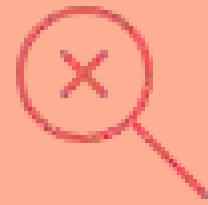
Resampled each image's sequential data to the median sequence length using **linear interpolation**, ensuring uniformity across samples.

```
median_length = int(np.median(df['Count']))

def resample_signal(signal, target_length):
    original = np.arange(len(signal))
    target = np.linspace(0, len(signal)-1, target_length)
    return interp1d(original, signal, kind='linear')(target)

df['Signal'] = df['Signal'].apply(lambda s: resample_signal(s, median_length))
```

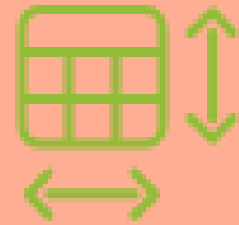

3. REMOVING NAN S



Missing values were identified and imputed using **linear interpolation** to ensure temporal continuity and preserve signal integrity.

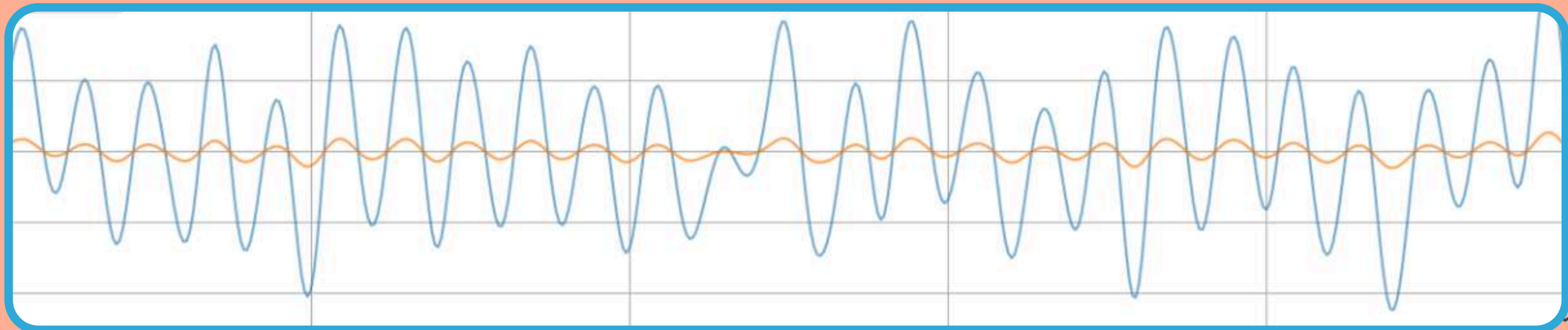
```
def interpolate_nans(epoch):  
    for i in range(epoch.shape[0]):  
        if np.isnan(epoch[i]).any():  
            x = np.arange(epoch.shape[1])  
            y = epoch[i]  
            nan_mask = np.isnan(y)  
            y[nan_mask] = np.interp(x[nan_mask], x[~nan_mask], y[~nan_mask])  
    return epoch  
  
epochs_data = np.array([interpolate_nans(e) for e in epochs_data])
```

4. NORMALISATION

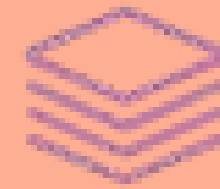


Applied **z-score normalisation** to the frequency data, scaling values to the $[-1, 1]$ range. It enables **consistent scale** across samples and **reduces noise** from irrelevant variability (e.g., individual differences in skull thickness).

```
def zscore_normalize(eeg):  
    mean = np.mean(eeg, axis=1, keepdims=True)  
    std = np.std(eeg, axis=1, keepdims=True)  
    return (eeg - mean) / (std + 1e-8) # small epsilon for numerical stability
```



5. PADDING TO 128 CHANNELS

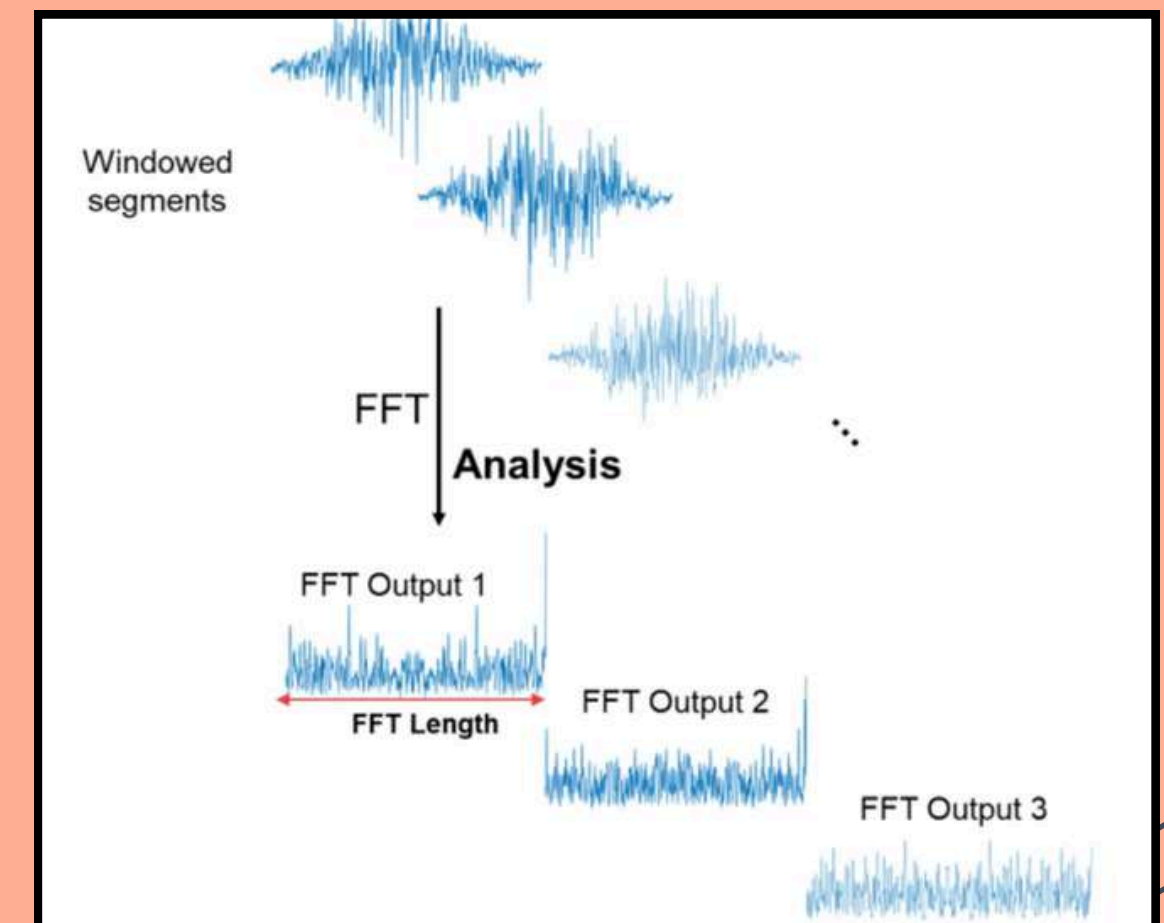
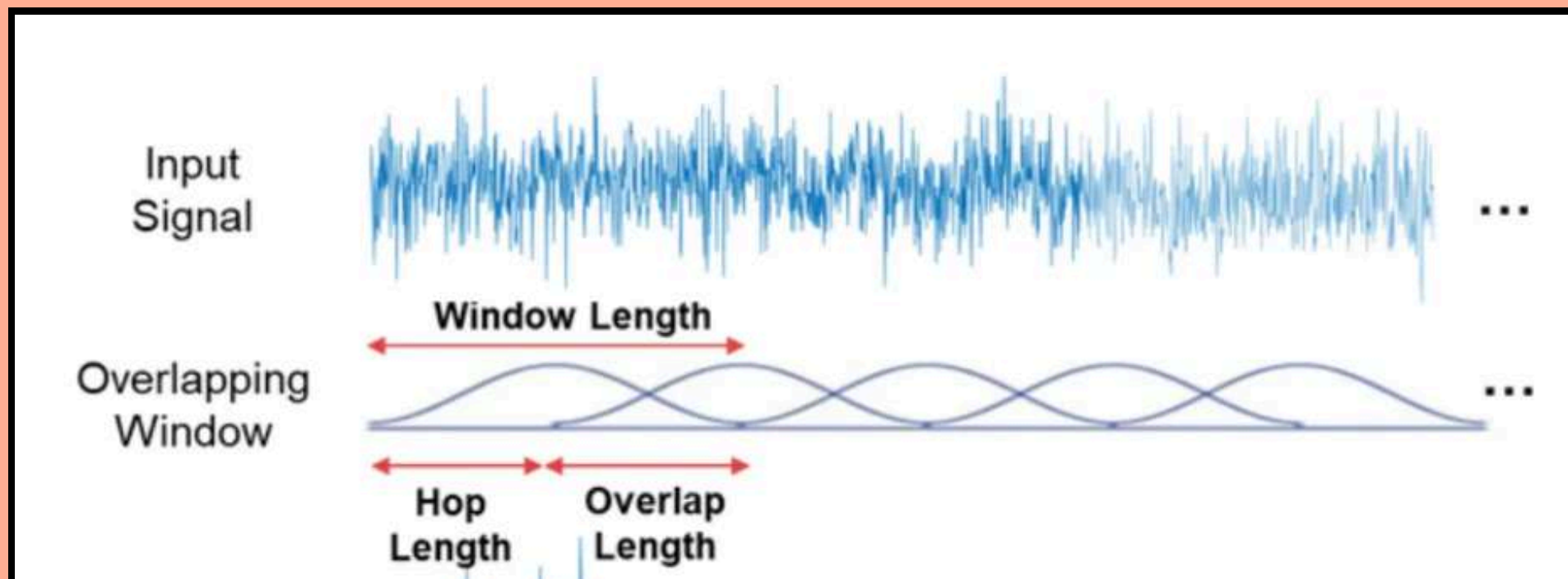


All samples were checked for incomplete channel data and standardized to 128 channels through duplication of existing channels where necessary.

```
def pad_channels_to_128(epoch):  
    num_channels = epoch.shape[0]  
    if num_channels == 128:  
        return epoch  
    elif num_channels < 128:  
        reps = 128 // num_channels + 1  
        padded = np.tile(epoch, (reps, 1))[:128, :]  
        return padded  
    else:  
        return epoch[:128, :]
```


FOURIER TRANSFORM

- Converts the signal from the time domain into the frequency domain
- The **FFT(Fast Fourier Transform)** is computed on overlapping windowed segments of the signal, and we get what is called the **spectrogram**



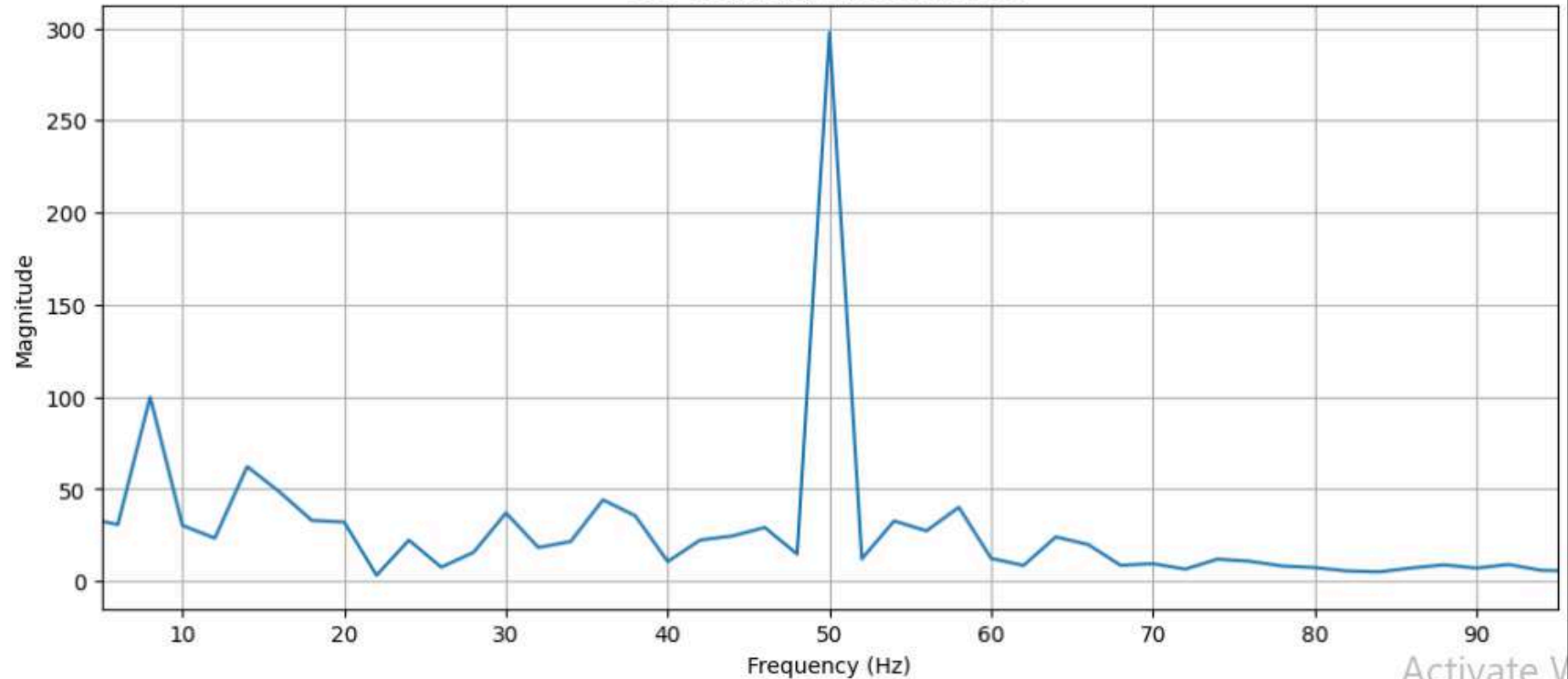
APPLYING FFT

```
n = len(eeg_signal) # Number of samples
fft_result = np.fft.fft(eeg_signal) # Compute FFT
frequencies = np.fft.fftfreq(n, d=1/fs) # Frequency bins

# Take only the positive frequencies (since FFT is symmetric)
half_n = n // 2
positive_freq = frequencies[:half_n]
magnitude = np.abs(fft_result[:half_n]) # Magnitude spectrum

plt.figure(figsize=(12, 5))
plt.plot(positive_freq, magnitude)
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('EEG Frequency Spectrum (FFT)')
```

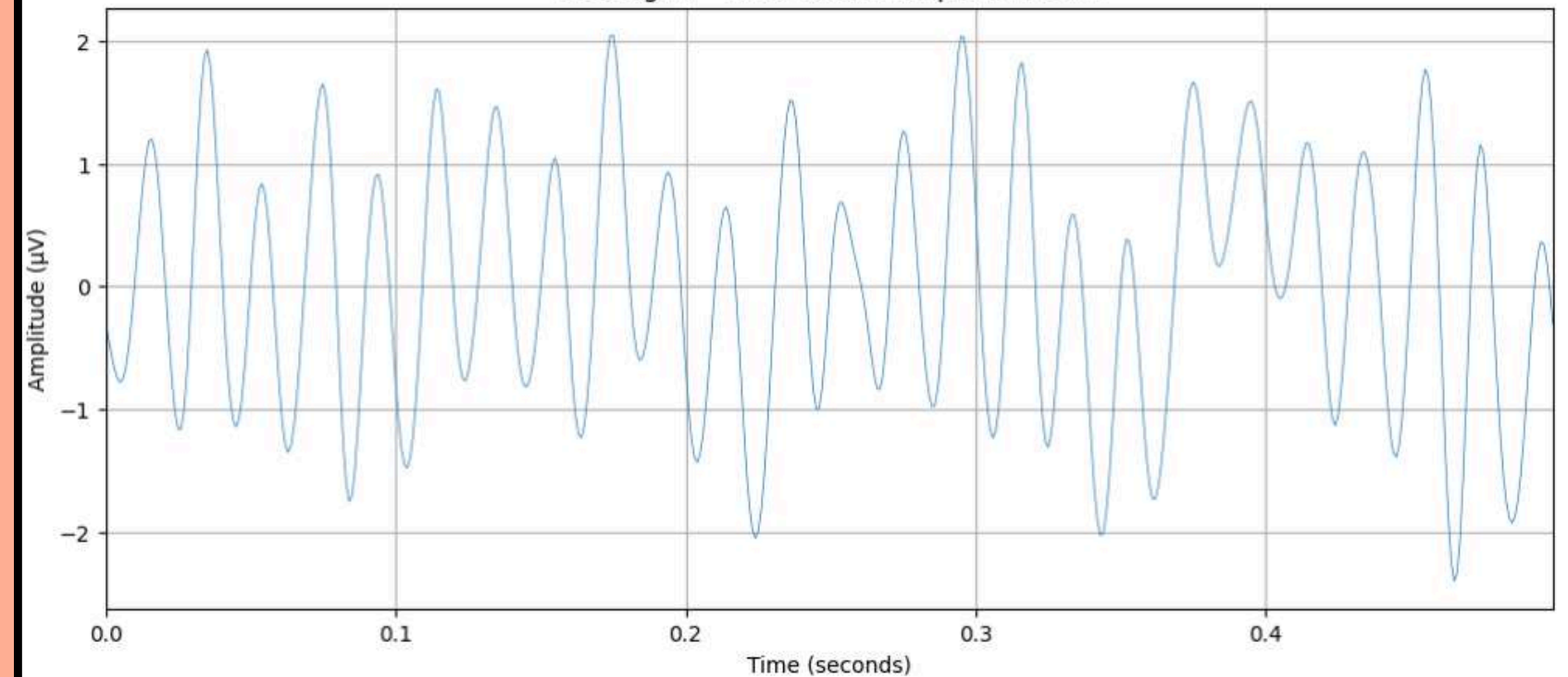
EEG Frequency Spectrum (FFT)



```
eeg_signal = data['eeg'][0][5] # Your EEG time series
n_samples = len(eeg_signal)
time = np.arange(n_samples) / fs # Time in seconds

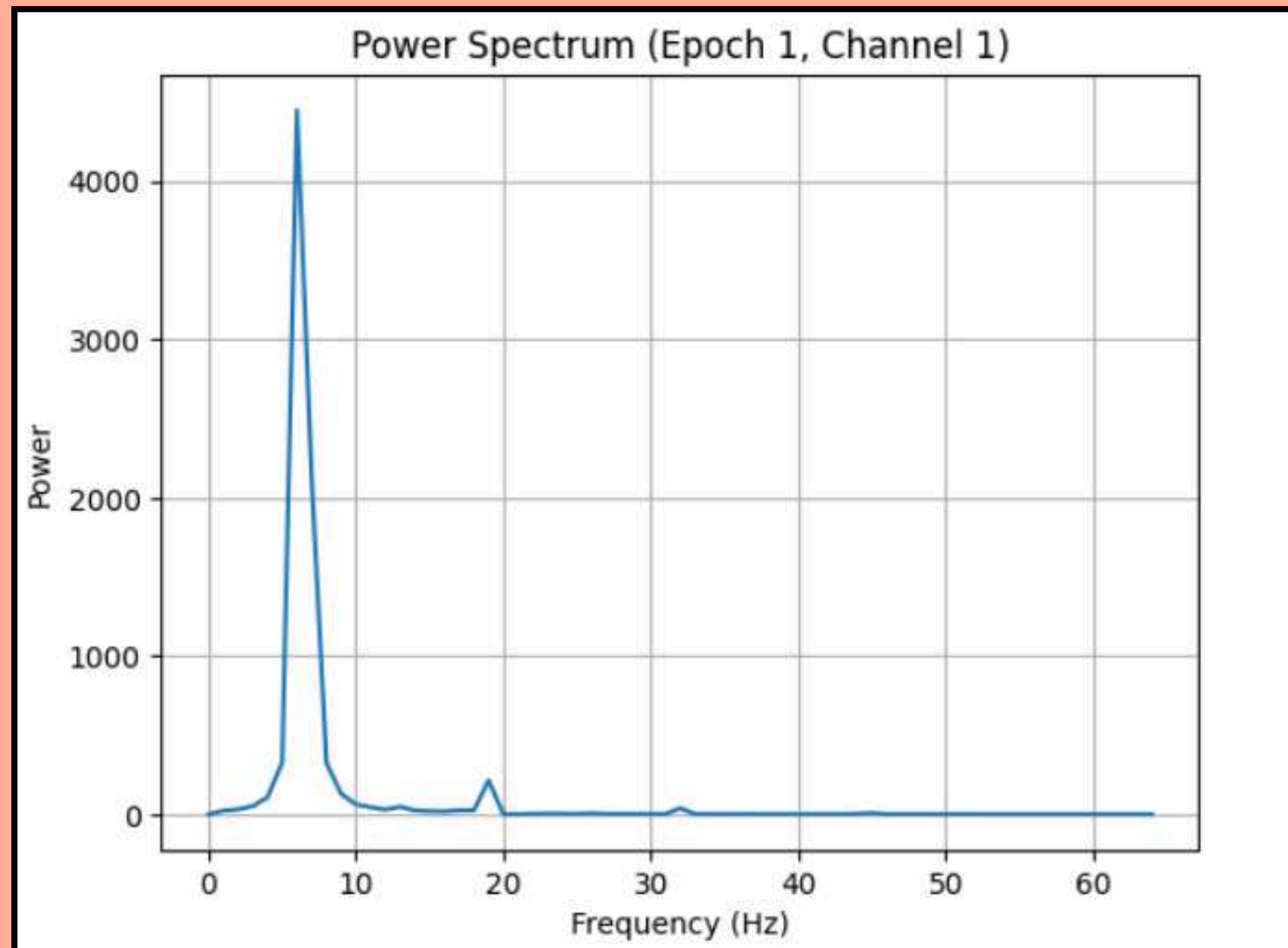
# Plot settings
plt.figure(figsize=(12, 5))
plt.plot(time, eeg_signal, linewidth=0.5) # Thin line for dense EEG data
plt.xlim([time[0], time[-1]]) # Show full duration
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude (μV)') # Assuming EEG is in microvolts
plt.title('EEG Signal - Time Domain Representation')
```

EEG Signal - Time Domain Representation



POWER SPECTRUM

```
def apply_fft(eeg_data, sfreq=128):  
    fft_vals = np.fft.rfft(eeg_data, axis=-1) # (epochs, channels, freqs)  
    power = np.abs(fft_vals) ** 2  
    freqs = np.fft.rfftfreq(eeg_data.shape[-1], d=1/sfreq)  
    return power, freqs
```



Power spectrum is computed using the basic **Periodogram method** via FFT.

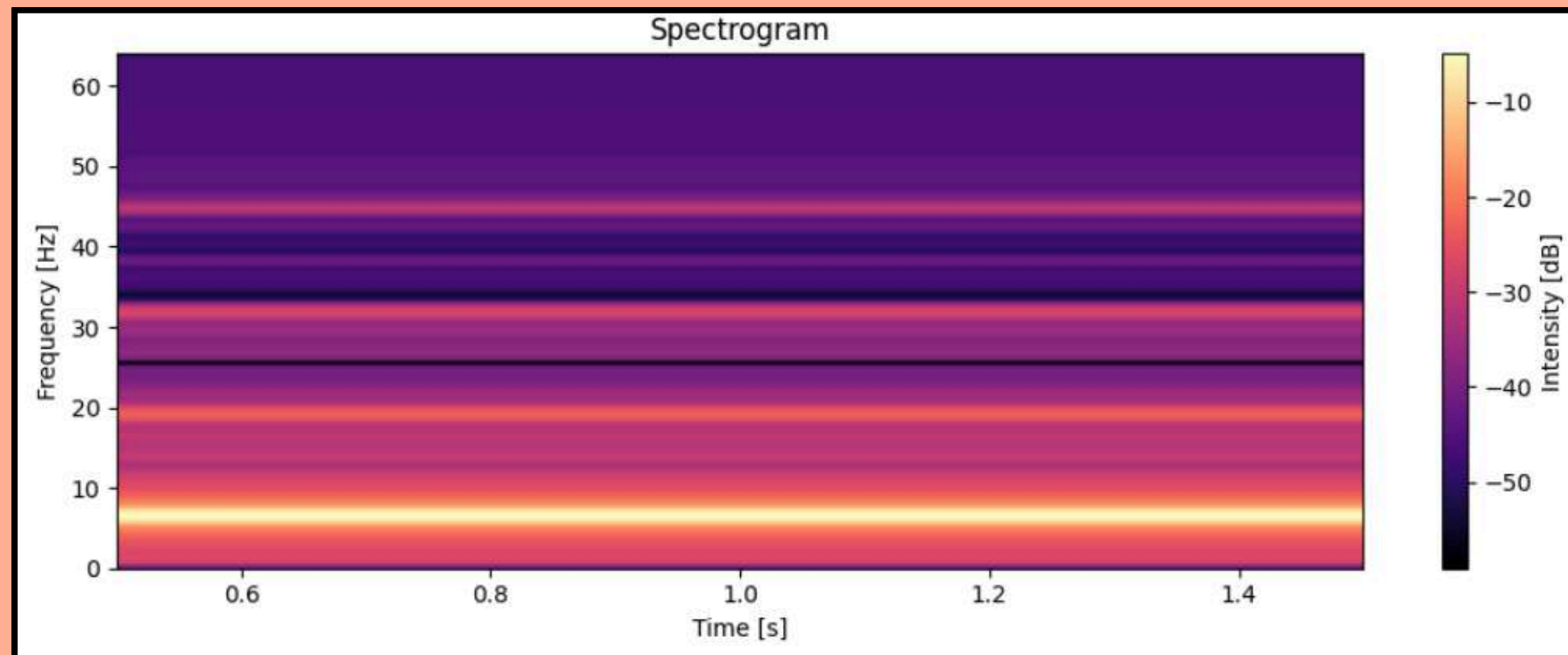
$$P(f) = |\text{FFT}(x)|^2$$

Gives a quick estimate but may be **noisy**, especially with short or non-stationary signals.

SPECTROGRAM

Applying
plt.specgram

```
def plot_spectrogram(eeg_data, sfreq=128, channel_idx=0):  
    signal = eeg_data[channel_idx] # shape: (n_times,)   
    plt.figure(figsize=(10, 4))  
    plt.specgram(signal, NFFT=256, Fs=sfreq, noverlap=128, cmap='magma')  
    plt.title("Spectrogram")  
    plt.xlabel("Time [s]")  
    plt.ylabel("Frequency [Hz]")  
    plt.colorbar(label='Intensity [dB]')  
    plt.tight_layout()  
    plt.show()
```



Spectrogram for
Channel 1

TEMPORAL TOKENISATION

- Every 4 adjacent time steps were grouped into a **token**, given the high temporal resolution. The sequence length is reduced while **temporal context** is preserved.
- For subsequent masked modelling, each token was then **transformed** into a **1024-dimensional embedding** through a projection layer.

```
def tokenize_sample(eeg_sample, token_length=4):
    C, T = eeg_sample.shape

    pad_length = (token_length - (T % token_length)) % token_length
    if pad_length > 0:
        padded_data = np.pad(eeg_sample, ((0, 0), (0, pad_length)), mode='edge')
    else:
        padded_data = eeg_sample

    num_tokens = padded_data.shape[1] // token_length
    tokens = padded_data.reshape(128, num_tokens, token_length) # (128, num_tokens, 4)
    tokens = np.transpose(tokens, (1, 0, 2)) # (num_tokens, 128, 4)

    return tokens
```

```
projection = nn.Linear(128 * 4, 1024)

def embed(tokens):
    all_embeddings = []
    tokens = torch.tensor(tokens, dtype=torch.float32)
    flattened = tokens.reshape(-1, 128 * 4) # (num_tokens, 512)
    embeddings = projection(flattened)
    all_embeddings.append(embeddings)
    return all_embeddings
```

WHY MASK ?

1) Self-supervised learning:

Masking enables self-supervised learning by **training** the model **to reconstruct masked parts** of the signal from the unmasked parts, similar to **how BERT masks words in NLP**.

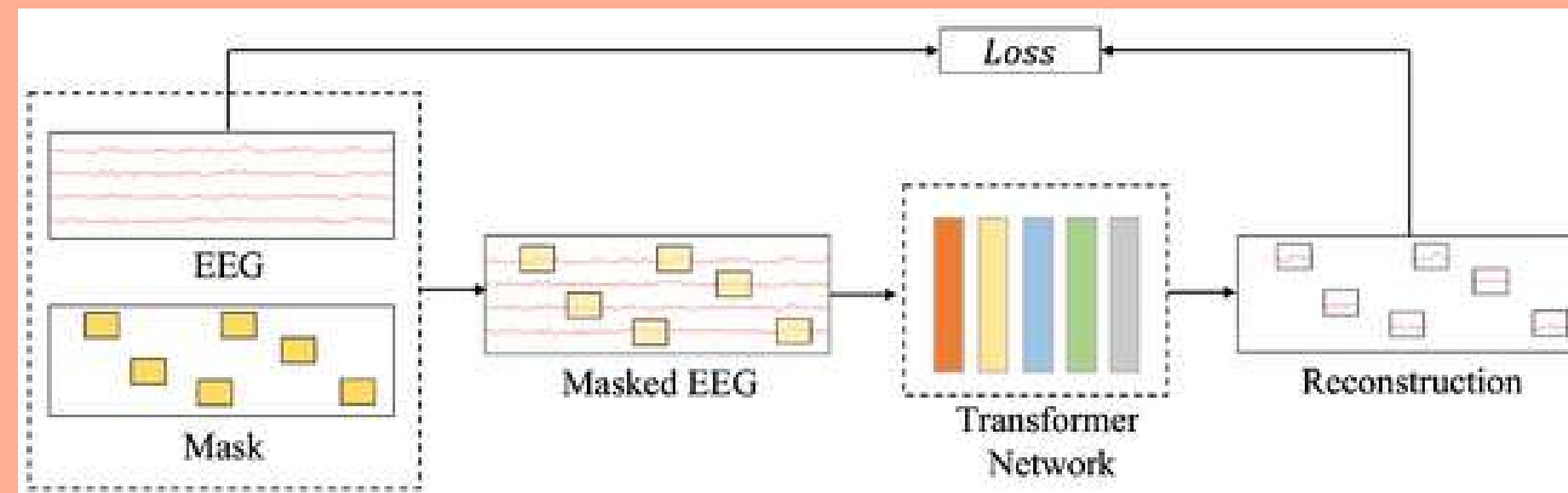
2) Learning Robust Representations:

Masking forces the model to understand the structure and dynamics of EEG signals instead of memorising them.

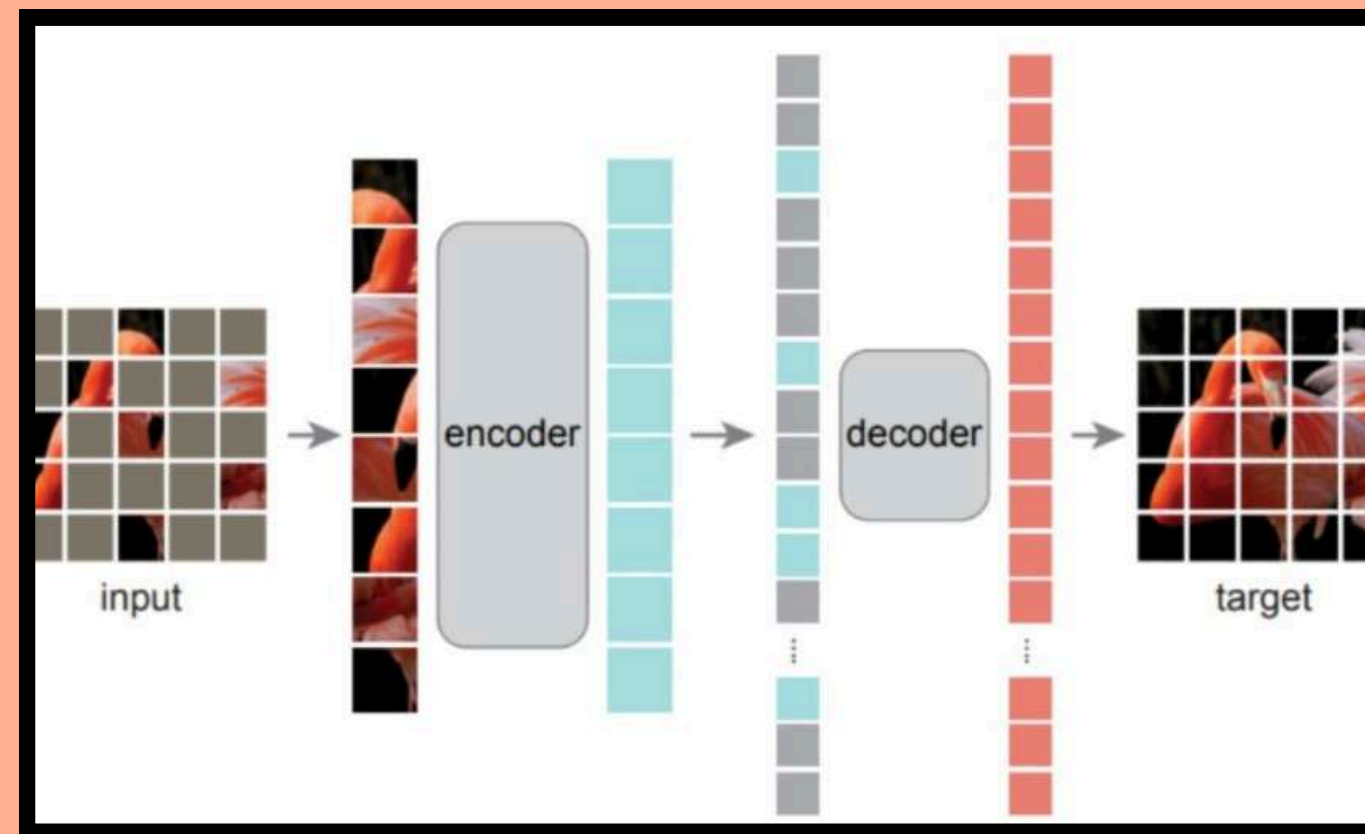
This encourages the model to **capture global patterns** (like frequency rhythms or cognitive features) that generalise better to downstream tasks, such as image reconstruction.

3) Avoiding Overfitting:

Masking acts like a **regularizer**, reducing overfitting by preventing the model from relying on local noise.

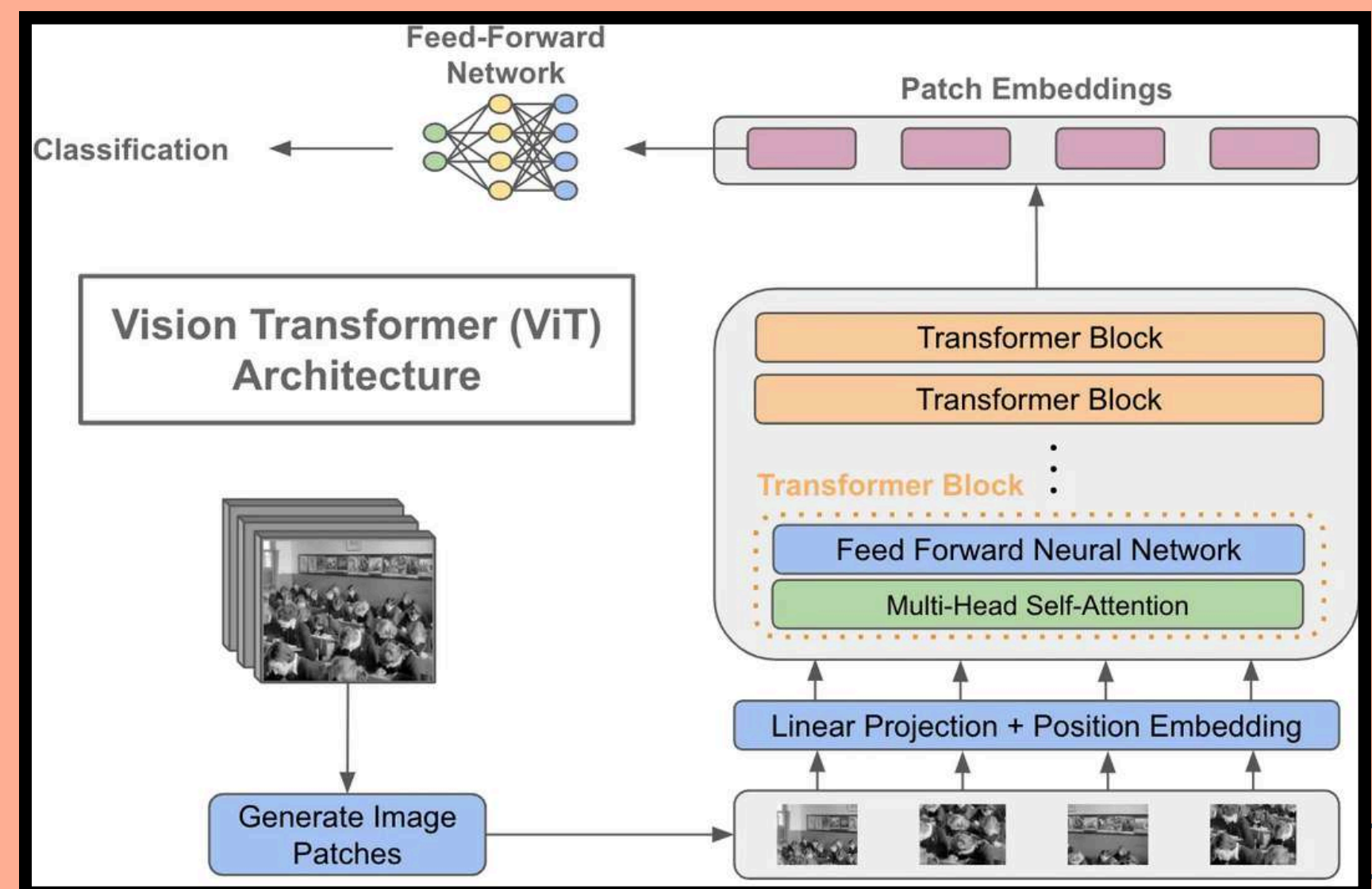


MODELING: MASKED AUTOENCODER APPROACH



- A large fraction of tokens are **masked**.
- Only the **visible tokens** are fed in the **encoder** part of MAE.
- The **decoder reconstructs** the masked tokens to predict the input signal.
- The model is forced to learn **contextual** and **global temporal** relationships.

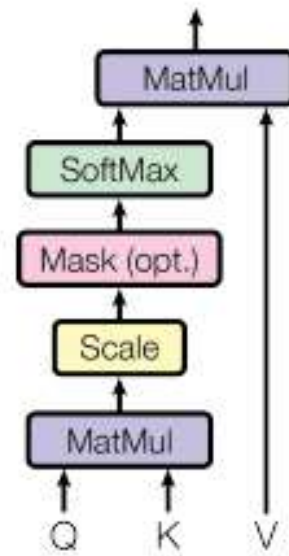
- MAE works on the **ViT (Vision Transformer)** backbone.
- **Positional encodings** are added to each token's vector (**patch embeddings** along with **classification [CLS] tokens**) to retain the **temporal** and **spatial** order information essential for EEG sequence modeling.



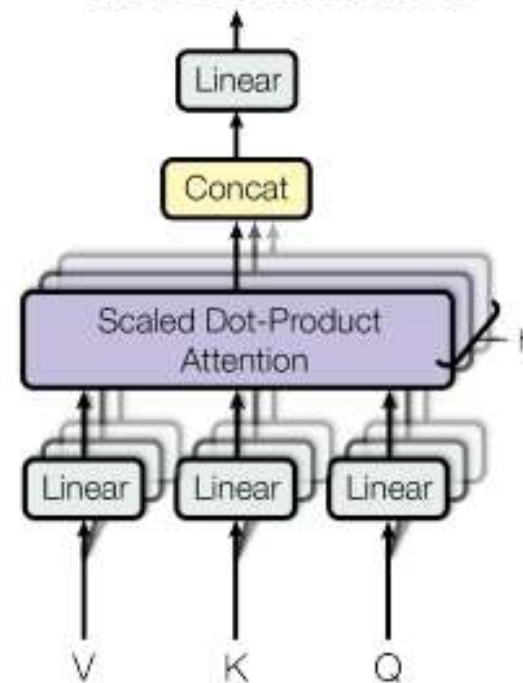
IS ATTENTION WHAT WE REALLY NEED??



Scaled Dot-Product Attention



Multi-Head Attention



When he asks you how much attention do you need???



TRANSFORMER

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

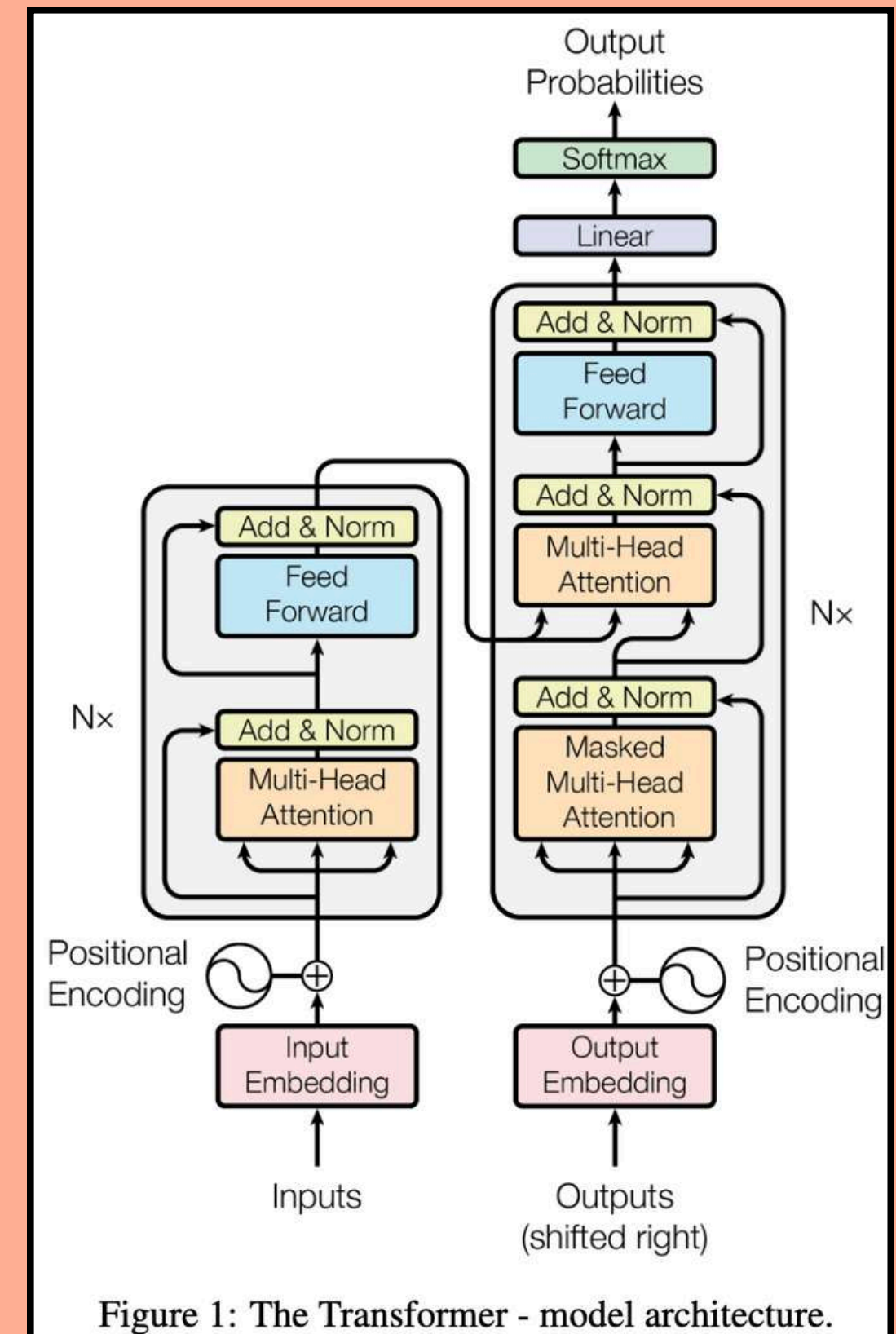
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

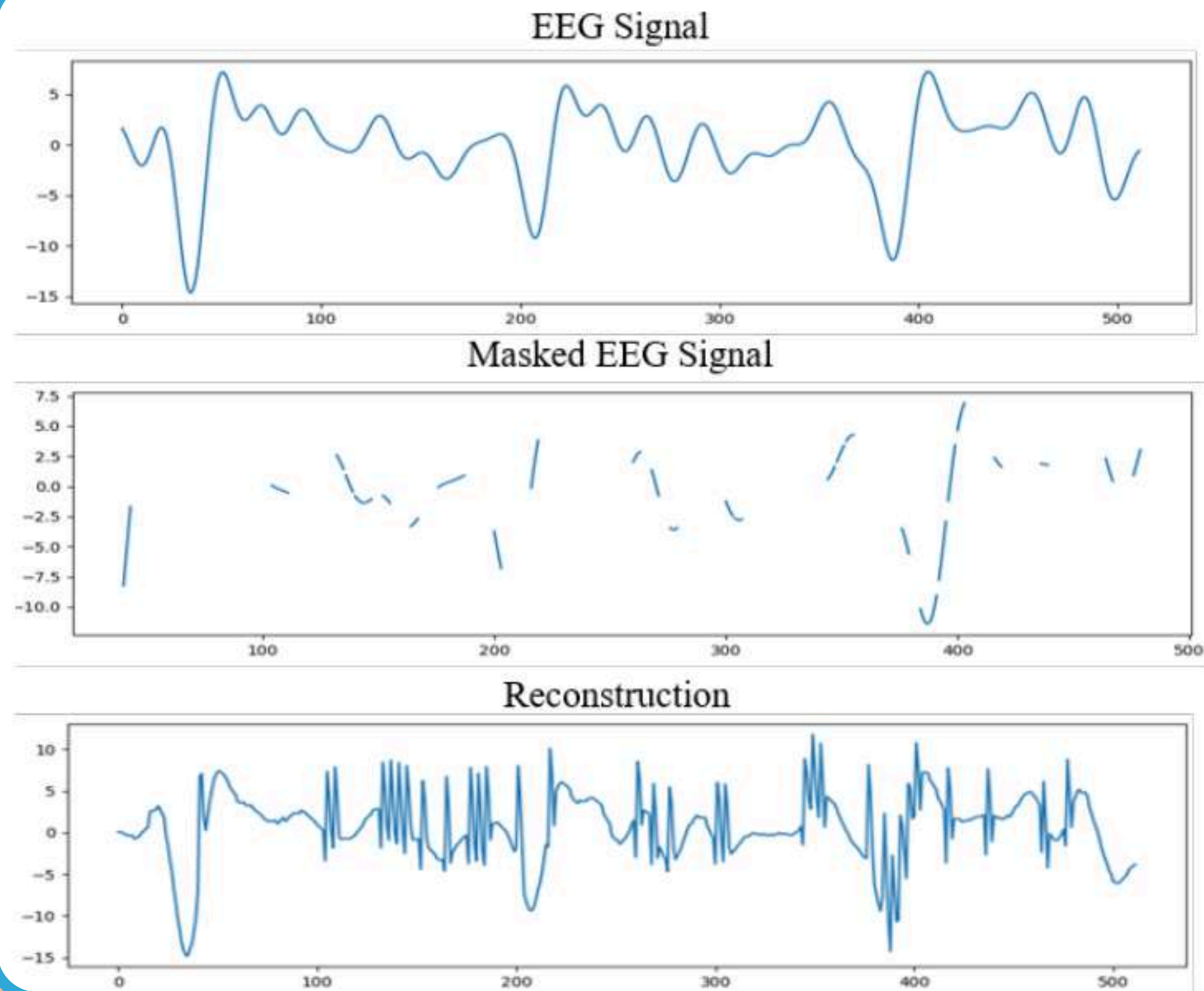
[illegible]

TRANSFORMER ARCHITECTURE

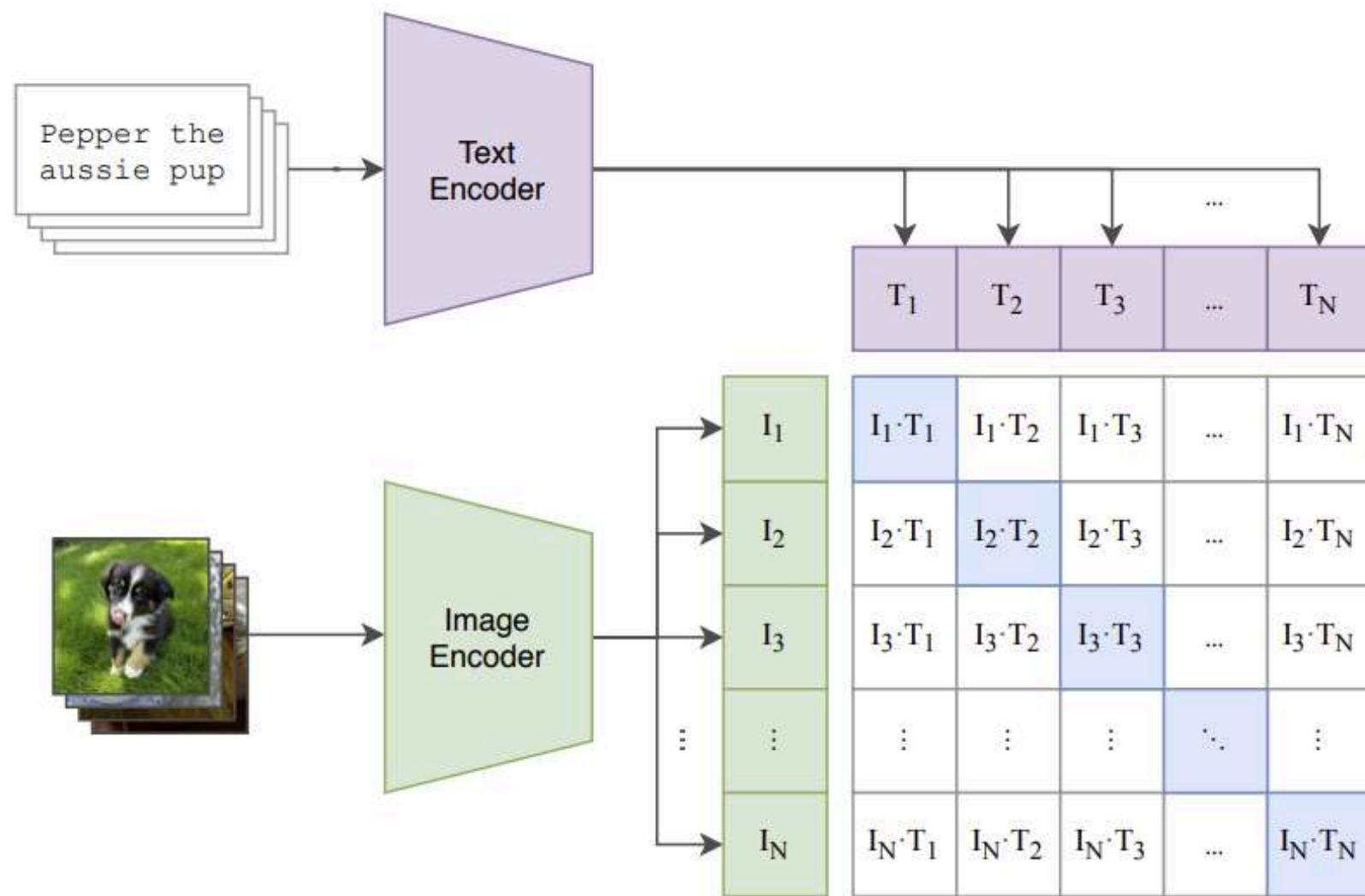
- The first architecture to take in **all time-steps** of a input sequence simultaneously and return the prediction of the output sequence. This was all possible due to the introduction of '**Attention**' Mechanism.
- This provided advantage over the traditional time dependent architectures like **RNN, LSTM, Bi-LSTM** which could only take one time-step at a time as input.
- **Transformer** proved to be a better Seq2Seq Model while capturing context over a **long span of time and data space in all directions**-- All Thanks to **ATTENTION!!!** :)



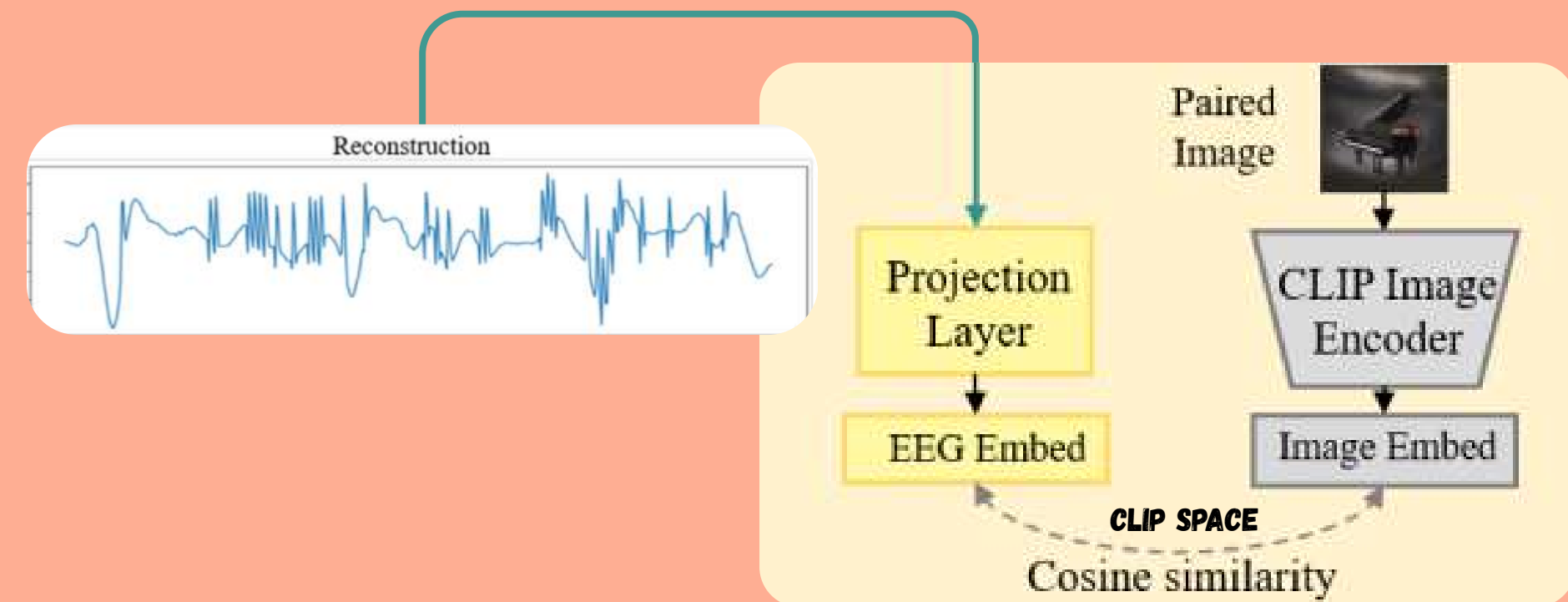
RESULTS FROM MAE MODEL



CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING



- Contrastive learning maps the image and EEG embeddings from both encoders into a joint or shared embedding space.

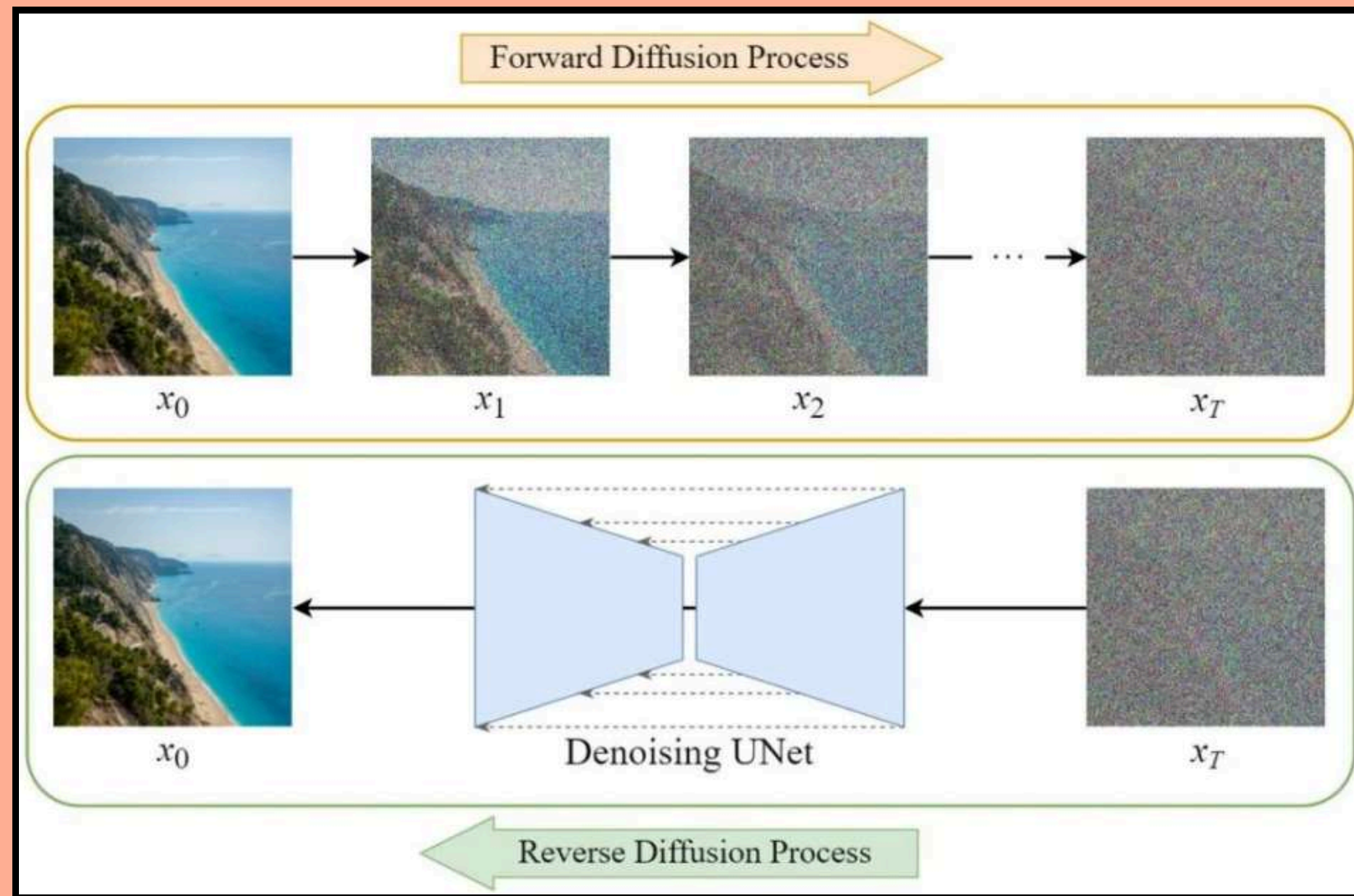


- The images are projected into the CLIP embedding space to generate their corresponding CLIP image embeddings.
- We trained the mapping from EEG to CLIP text space using a cosine similarity loss between the EEG embeddings and their corresponding CLIP image embeddings.

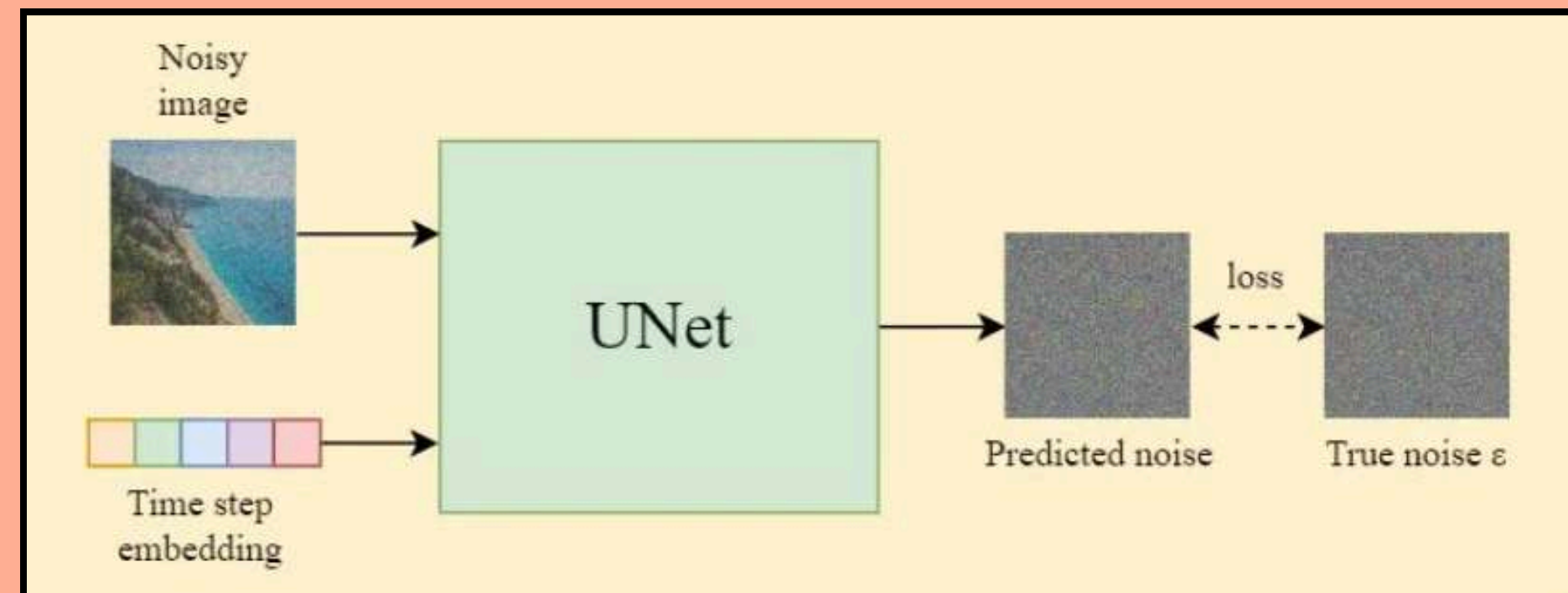
STABLE DIFFUSION 1.5

- Stable Diffusion is a type of generative model that creates images by gradually “denoising” random noise into a coherent image based on the input text

Diffusion Model

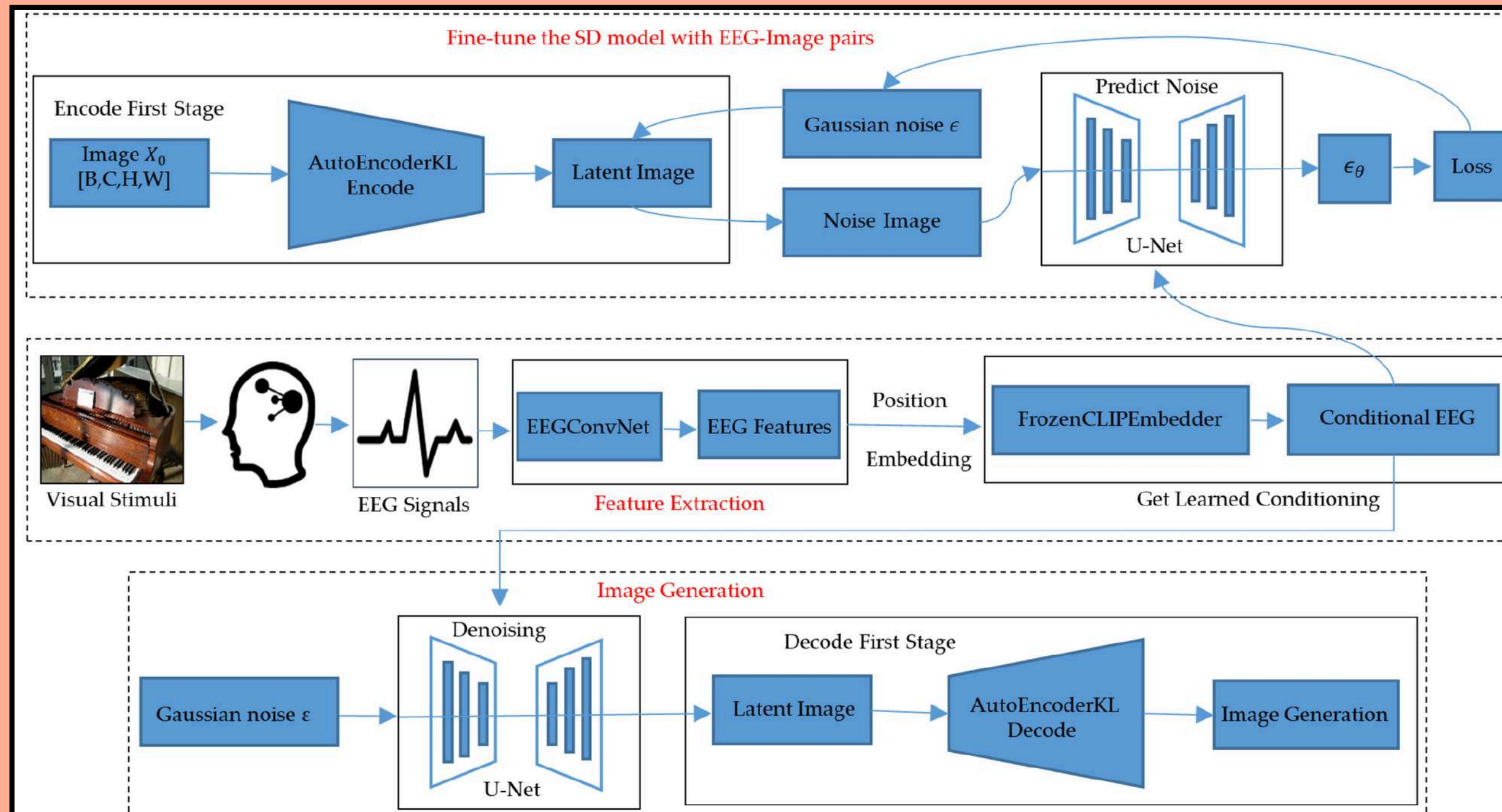


Training SD to predict noise



FINE-TUNING STABLE DIFFUSION 1.5

Using SD 1.5 to generate images from the CLIP embeddings



- Conditional signals are introduced by the cross-attention mechanism in the U-Net. This cross-attention can also incorporate conditional information from the EEG data

- We optimize the EEG encoder and cross-attention heads of the U-Net together. We keep the remaining parts of Stable Diffusion fixed.

METRIC FOR FINAL EVALUATION

```
from PIL import Image
import torch
import torch.nn.functional as F
import torchvision.transforms as transforms

def load_image(path):
    img = Image.open(path).convert("RGB") # Works for PNG, JPEG, etc.
    transform = transforms.Compose([
        transforms.Resize((450, 326)), # Ensure same size
        transforms.ToTensor(), # Convert to Tensor [0,1]
    ])
    return transform(img).view(-1) # Flatten to 1D vector

# Paths to your images (can be JPEG or PNG)
img_path1 = "/kaggle/input/eeg-code1/imageNet_images/n03376595/n03376595_247.JPEG"
img_path2 = "/kaggle/working/generated_image.png"

# Load as tensors
img_tensor1 = load_image(img_path1).to(torch.float32)
img_tensor2 = load_image(img_path2).to(torch.float32)

# Compute cosine similarity and loss
cosine_sim = F.cosine_similarity(img_tensor1.unsqueeze(0), img_tensor2.unsqueeze(0))
cosine_loss = 1 - cosine_sim

print("Cosine Similarity:", cosine_sim.item())
print("Cosine Loss:", cosine_loss.item())
```

Cosine Similarity: 0.7767267227172852
Cosine Loss: 0.22327327728271484

Average Cosine
Similarity of all the
results = 0.65



INFERENCE DEMONSTRATION

```
image = var.decode(latent / 0.10210).sample()

image = (image.clamp(-1, 1) + 1) / 2
return image.cpu()

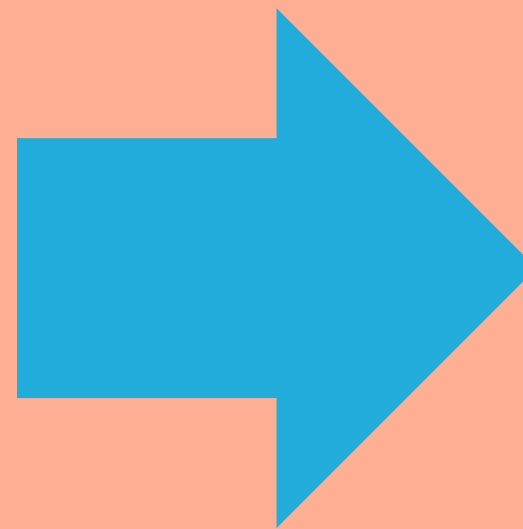
-----
Generate and Display Image
-----
-----
Assume you already imported everything and loaded models
-----

-----
EEG 1D Reshape Utility (NEW)
-----
def reshape_1d_eeg(eeg_1d, num_channels=128):
    eeg_1d = np.array(eeg_1d)
    total_points = eeg_1d.shape[0]
    pad_len = (num_channels - (total_points % num_channels)) % num_channels
    if pad_len > 0:
        eeg_1d = np.pad(eeg_1d, (0, pad_len), mode='edge')
    reshaped = eeg_1d.reshape(num_channels, -1)
    return reshaped

-----
Use a sample EEG and image
-----
sample_idx = 0
```

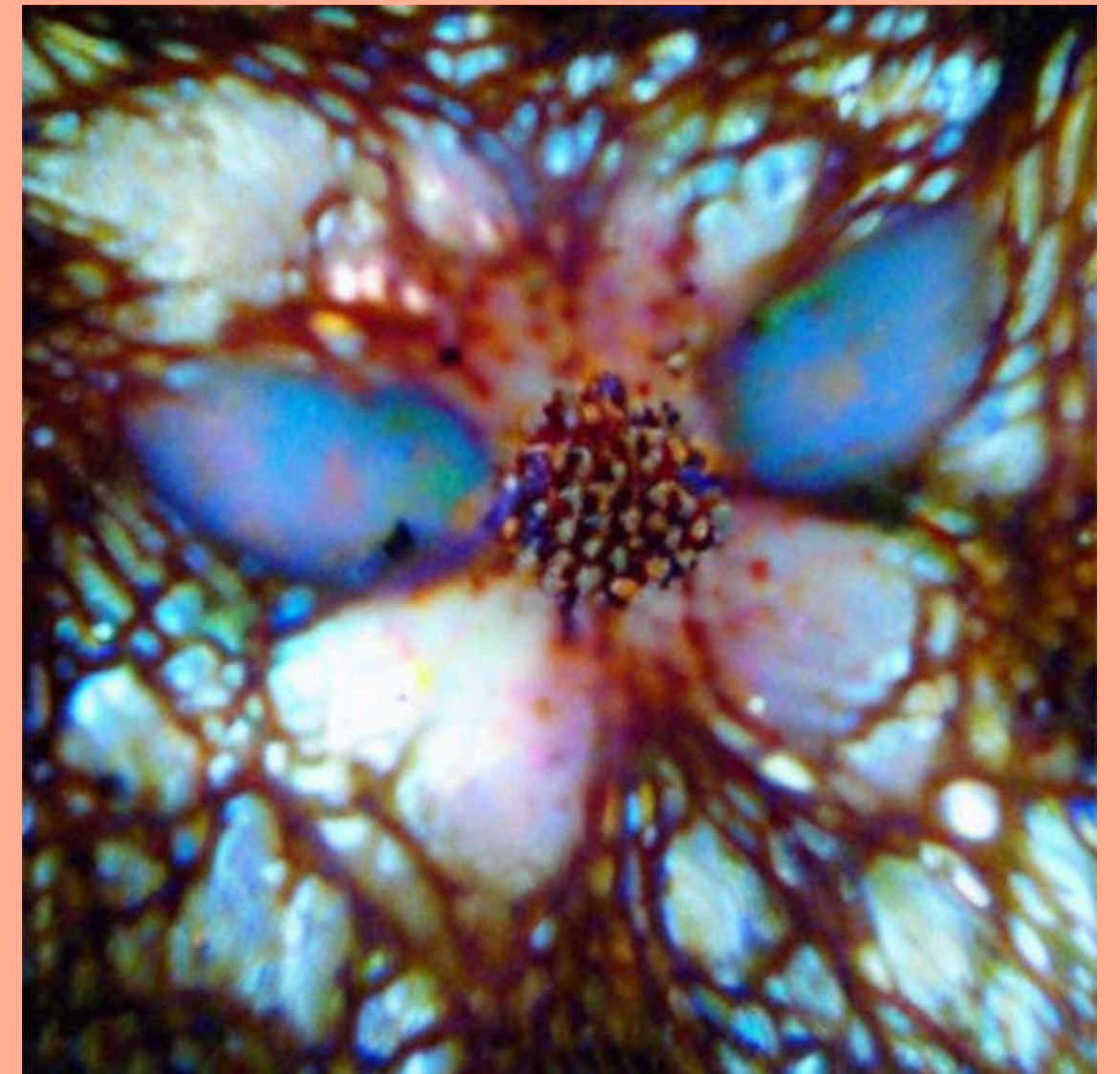
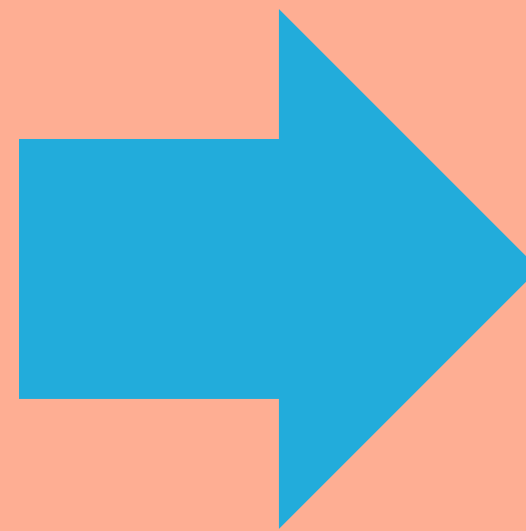
RESULTS

These are the images we generated using our fine-tuned Stable Diffusion Model



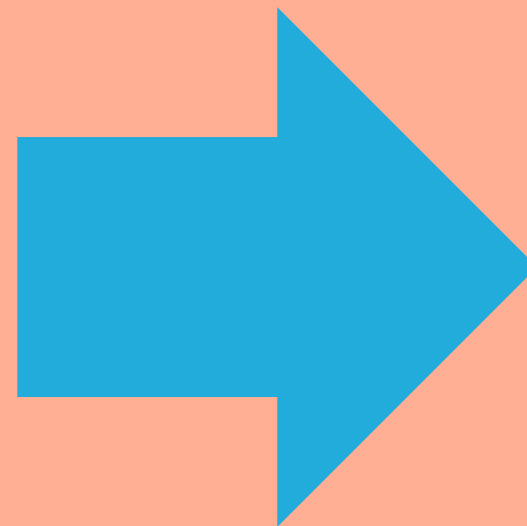
RESULTS

These are the images we generated using our fine-tuned Stable Diffusion Model



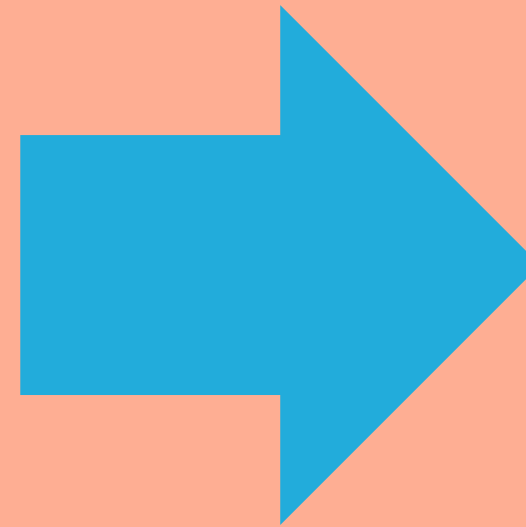
RESULTS

These are the images we generated using our fine-tuned Stable Diffusion Model



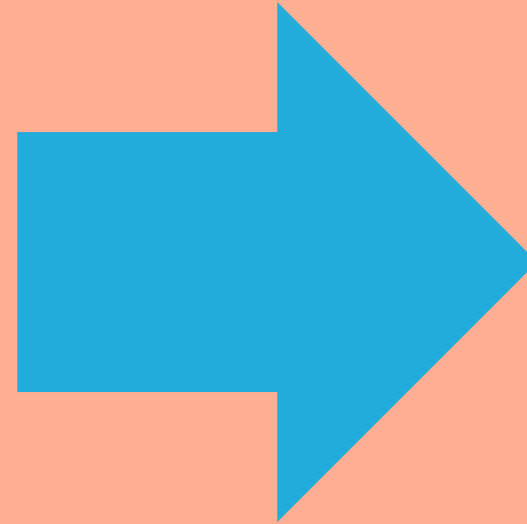
RESULTS

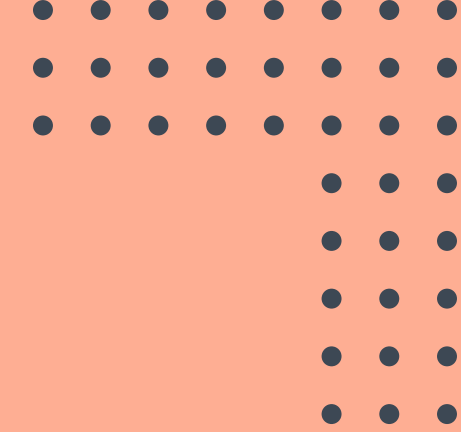
These are the images we generated using our fine-tuned Stable Diffusion Model



RESULTS

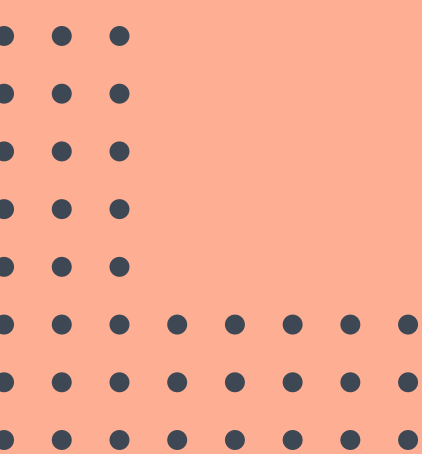
These are the images we generated using our fine-tuned Stable Diffusion Model





CONCLUSIONS

The Dream Diffusion Project successfully generates images that almost closely resembles those seen by users, using EEG-derived embeddings to condition a fine-tuned Stable Diffusion model. The visual outputs align well with the original stimuli, showcasing the effectiveness of our approach in mapping brain activity to images. These results not only validate our method but also open up exciting future directions in neural decoding, dream reconstruction, and brain-computer interfaces.



MENTORS



Akshay
Reddy



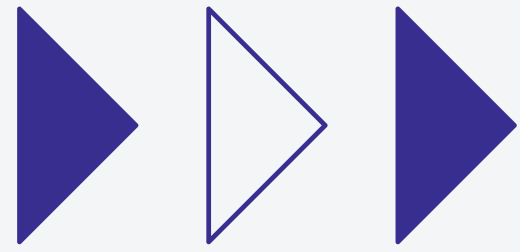
Akshat
Saxena



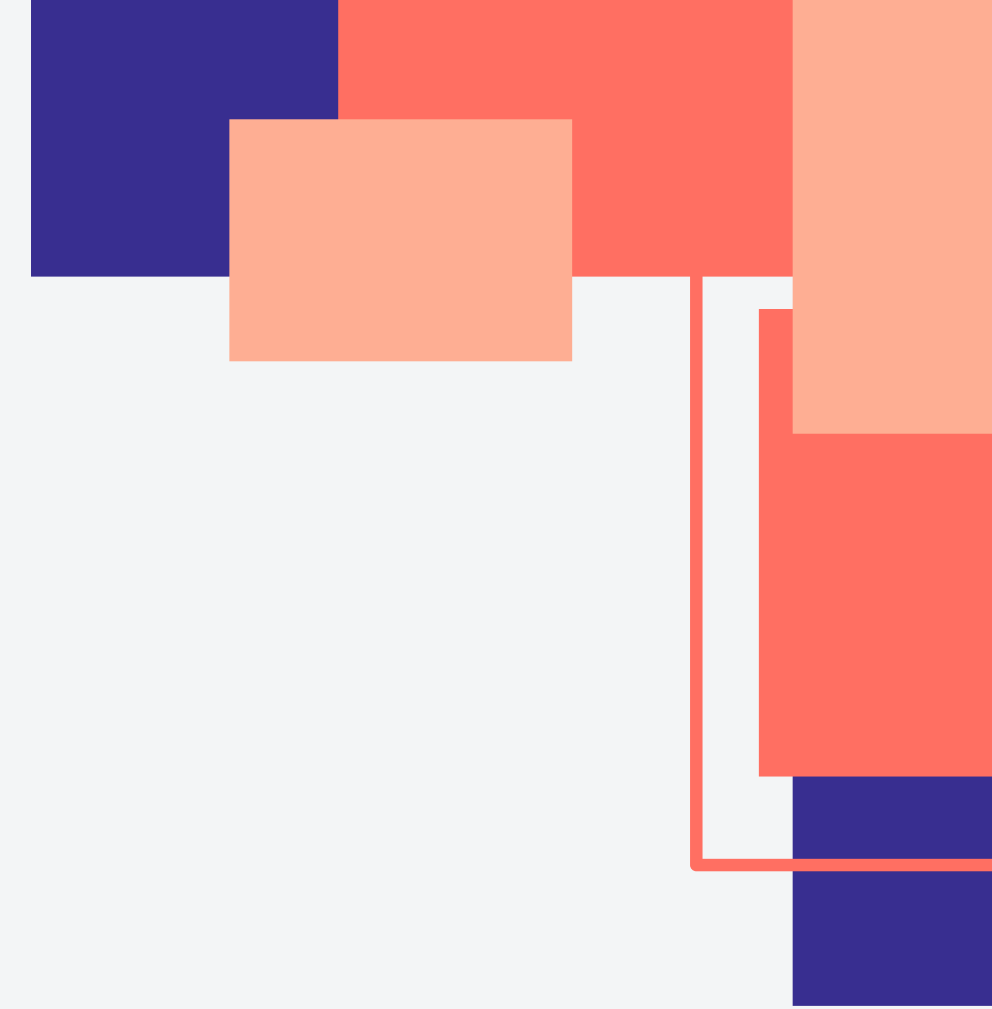
Akshat
Srivastava



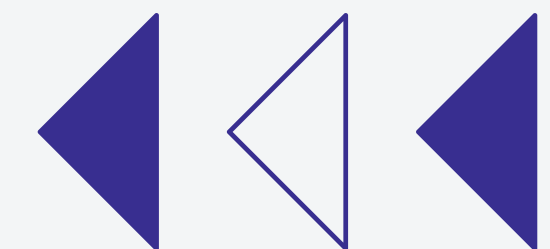
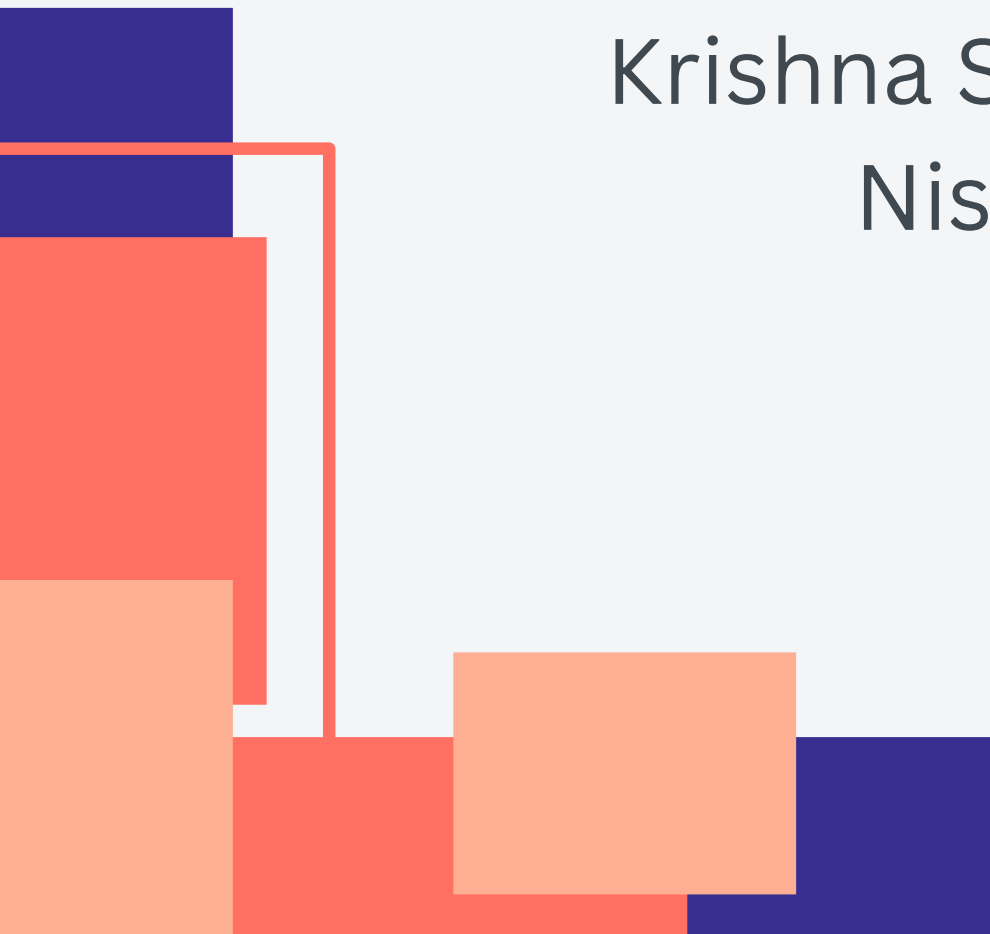
Hitarth
Makawana



MENTEES



Aditya Goel, Archisman Dhar, Astha Yadav,
Avtansh Gargya, Dhruv Kejriwal, Hirani Shah,
Joel John Mathew, Kartikey Tomar, Krishna Agrawal,
Krishna Sharma, Kuldip Manvar, Manthan J, Mayank Agarwal,
Nishant Dwivedi, Rudra Jain, Sahaj, Smira Jaitley,
Surya Prakash Pingali, Vaibhav Kalyan





THANK YOU